

Engineering Exercise Book

Dual-MCU IoT Sensor Node — Phases 0 to 4

~75 Exercises · Embedded-Context Specific · Solutions with Hardware-Level Explanations

STM32F411 Black Pill + ESP32 DevKit V1 · Fedora Linux

Exercise Book v1.0

How to Use This Book

Every exercise maps directly to a subphase in the masterplan. Work through them in order — each one builds on the previous. All exercises are **embedded-context specific**: no generic 'what does printf do' questions. Every problem relates to your STM32F411 Black Pill, your BMP280, your FreeRTOS task architecture, or your OTA pipeline.

Rule: Attempt every exercise before reading the solution. The solutions section is at the back of this book. A wrong answer with written reasoning is more valuable than reading the solution cold.

For C exercises (Phase 0–1): Compile on Fedora: `gcc -O0 -Wall -o out ex.c && ./out` Test logic on PC first, then port to the MCU.

Exercise Index

Phase	Topic	Exercises	Difficulty
0.1	C Language Hardening	7	Beginner–Intermediate
0.2	Makefile + Toolchain	5	Beginner
0.3	Datasheet Navigation	5	Beginner–Intermediate
1.1	GPIO + RCC	5	Intermediate
1.2	UART + Ring Buffer	5	Intermediate
1.4	I2C Driver + BMP280 + SSD1306	5	Intermediate–Advanced
1.5	SPI Driver	5	Intermediate
1.6	Timers + HC-SR04 + DHT22	5	Intermediate–Advanced
2A	FreeRTOS Task Architecture	6	Advanced
2B	ESP32 + MQTT	4	Intermediate–Advanced
3	Inter-MCU Protocol Design	4	Advanced
4	Bootloader + OTA Pipeline	5	Advanced

Phase 0.1 C Language Hardening

Compile on your Fedora machine: `gcc -O0 -Wall -o out ex.c && ./out` — no MCU needed yet.

E0.1-01 — Bitwise SET — The RCC Enable Pattern

Context: Every STM32 peripheral driver opens with this exact pattern: enable the clock in an RCC register without disturbing other bits.

Question

Write a C expression that sets bit 0 of `uint32_t` `reg` without changing any other bits. Then write the expression that sets bits 2 and 5 simultaneously in the same variable. Show both and print the result.

Hint: Use `|=` and `(1U << n)` to build masks.

E0.1-02 — Bitwise CLEAR — The GPIO Configuration Pattern

Context: To change a GPIO pin mode you first clear the relevant bits, then OR in the new value. Skipping the clear corrupts adjacent pins.

Question

Given `uint32_t` `moder = 0x0000FFFF`; (all pins 0–7 configured as outputs), write the C expression that clears bits [11:10] (pin 5 mode) WITHOUT affecting any other bit. Then set those same bits to 0b01 (output mode).

Hint: Clear with `&= ~(mask)`. Mask for 2 bits at position `n` is `(0x3U << n)`.

E0.1-03 — Bitwise READ — Checking Status Flags

Context: UART, I2C and SPI drivers constantly check status register bits to know when hardware is ready.

Question

Write a macro `IS_BIT_SET(reg, bit)` that evaluates to 1 if bit is set, 0 otherwise. Use it to write a busy-wait loop that waits until bit 6 (TXE flag) of `uint32_t` `USART_SR` is set. Then print how many iterations the loop ran.

Hint: Shift `1U` left by bit positions, AND with `reg`, compare `!= 0`.

E0.1-04 — volatile — Why the Compiler Lies Without It

Context: Missing volatile on peripheral register pointers causes drivers to break silently in optimised builds — a bug that only appears in production.

Question

Explain what the compiler does to this code at `-O2` and why it is wrong. Then fix it.

```
struct { uint32_t SR; } *usart = (void*)0x40004400;
```

```
while (!(usart->SR & (1<<6))) {}
```

Hint: What does the optimiser assume about memory that has no volatile qualifier?

E0.1-05 — Struct Pointer Casting — The Foundation of Peripheral Drivers

Context: This is exactly how your STM32 register access works. GPIOA lives at 0x40020000. Its registers are at fixed offsets from that base.

Question

Given:

```
typedef struct { volatile uint32_t MODER, OTYPER, OSPEEDR, PUPDR, IDR, ODR, BSRR; } GPIO_TypeDef;
#define GPIOA_BASE 0x40020000UL
```

Write C code to: (a) create a pointer to GPIOA, (b) set pin 5 as output (MODER[11:10]=01), (c) drive pin 5 HIGH using BSRR.

Hint: Cast GPIOA_BASE to GPIO_TypeDef*. BSRR lower 16 bits set pins, upper 16 bits clear pins.

E0.1-06 — Function Pointers — The Interrupt Vector Table

Context: The Cortex-M vector table is an array of function pointers. You must understand this syntax to write and debug interrupt handlers.

Question

(a) Write a typedef VoidHandler for a pointer to a function returning void with no arguments. (b) Write a 4-entry dispatch table where entry 0 is NULL, entries 1–3 point to UART_IRQHandler, I2C_IRQHandler, SPI_IRQHandler. (c) Write all three handler stubs.

Hint: typedef void (*TypeName)(void);

E0.1-07 — Challenge — Real STM32 Register Calculation

Context: This is the exact calculation you will perform at the start of Phase 1 for every GPIO pin.

Question

STM32F411 GPIOA_MODER reset value is 0xA8000000 (pins 13,14,15 in AF mode for SWD). Write C code to configure pins 0,1,2 as outputs and pin 3 as input, leaving pins 13,14,15 COMPLETELY unchanged. Pin N uses MODER bits [2N+1:2N].

Hint: Process each pin with clear-then-set. Input mode (00) only needs clear.

Phase 0.2 Makefile + Toolchain

Run in a terminal on Fedora. No MCU required.

E0.2-01 — Annotate Every Line of the Minimal Makefile

Context: You must be able to explain your entire Makefile before it becomes a black box.

Question

For each line of this Makefile write one sentence explaining what it does and why it exists:

```
TARGET=firmware

MCU=-mcpu=cortex-m4 -mthumb -mfpv4-sp-d16 -mfloat-abi=hard

CFLAGS=$(MCU) -O0 -g3 -Wall

LDFLAGS=$(MCU) -Tstm32f411.ld -Wl,--gc-sections

all: $(TARGET).bin

$(TARGET).elf: main.c

arm-none-eabi-gcc $(CFLAGS) $(LDFLAGS) -o $@ $^

$(TARGET).bin: $(TARGET).elf

arm-none-eabi-objcopy -O binary $< $@
```

Hint: \$@ = target name, \$^ = all prerequisites, \$< = first prerequisite.

E0.2-02 — Add size and clean Targets

Context: arm-none-eabi-size tells you how many bytes your code uses in flash and RAM — essential for verifying the bootloader fits in 32KB.

Question

Add a 'size' target and a 'clean' target to the Makefile from E0.2-01. Size must depend on the .elf. Clean must remove *.o *.elf *.bin *.map. Mark both .PHONY.

Hint: Targets that produce no file should be .PHONY so Make always runs them.

E0.2-03 — The Timestamp Dependency Bug

Context: A common Makefile mistake causes a full recompile on every 'make' even when nothing changed.

Question

This Makefile always recompiles everything even when sources are unchanged. Identify the bug and fix it:

```
all:

arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -o firmware.elf main.c
```

Hint: Make compares timestamps: target file vs prerequisite files. What file does 'all' produce?

E0.2-04 — Compiler Flag Purpose Quiz

Context: You must explain every flag in your compile command on demand — in an interview or when debugging a build failure.

Question

For each flag state: (a) what it tells the compiler/linker, (b) what breaks on STM32F411 if removed.

1. -mfpv4-sp-d16 2. -mfloat-abi=hard 3. -ffunction-sections 4. -Wl,--gc-sections 5. -specs=nano.specs

Hint: Think about FPU, calling convention, dead code, and C library size.

E0.2-05 — Linker Script Anatomy

Context: The linker script is the file that tells the linker where your flash and RAM are. Without it, your binary will not run on the MCU.

Question

Annotate every line of this minimal STM32F411 linker script — explain what each line does:

```
MEMORY {  
FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 512K  
RAM (rwx) : ORIGIN = 0x20000000, LENGTH = 128K  
}  
SECTIONS {  
.text : { *(.text*) } > FLASH  
.data : { *(.data*) } > RAM AT > FLASH  
.bss : { *(.bss*) } > RAM  
}
```

Hint: ORIGIN = start address, LENGTH = size. AT > means 'load from here but run in here'.

Phase 0.3 Datasheet Navigation

You need RM0383 open for these. Download free from st.com (search RM0383). All answers come from the document.

E0.3-01 — Memory Map — Peripheral Addresses

Context: You must know where peripherals live in memory before writing a single register access.

Question

Using RM0383 Section 2.3, find and record: (a) GPIOA base address, (b) USART1 base address, (c) I2C1 base address, (d) SPI1 base address, (e) TIM2 base address. For each state which bus it is on (AHB1, APB1, or APB2) and why the bus matters.

Hint: Table 1 in Section 2.3 lists all peripheral base addresses with bus information.

E0.3-02 — UART BRR Register Calculation

Context: This exact calculation is the first thing you write in your UART init function.

Question

Using RM0383 Section 30.3.4, calculate the BRR register value for 115200 baud on USART2 with PCLK1 = 42 MHz (OVER8 = 0). Show all working. What is the actual baud rate error?

Hint: $USARTDIV = PCLK / \text{baud}$. Integer part $\rightarrow$ bits [15:4]. Fraction $\times 16$, rounded $\rightarrow$ bits [3:0].

E0.3-03 — GPIO Configuration Bit Fields

Context: Every GPIO driver function requires these four register writes in the correct order.

Question

Using RM0383 Section 8.4, list the exact register, bits, and values to configure PA5 as: push-pull output, medium speed (25 MHz), no pull-up or pull-down.

Hint: Four registers: MODER, OTYPER, OSPEEDR, PUPDR. Each has a different encoding.

E0.3-04 — I2C CCR Calculation

Context: Wrong CCR causes I2C timing violations that corrupt data with no error message.

Question

Using RM0383 Section 27.6.8, calculate CCR for 100 kHz standard mode on I2C1 with PCLK1 = 42 MHz. Also calculate TRISE (Section 27.6.9). Show all working.

Hint: Standard mode: $CCR = PCLK1 / (2 \times f_{SCL})$. $TRISE = \text{ceil}(t_{\text{rise_max}} / t_{PCLK1}) + 1$. Standard max rise time = 1000 ns.

E0.3-05 — Cortex-M4 Exception Types

Context: When your code crashes you need to know which exception fired and why. The ARM User Guide describes all exception types.

Question

Using the ARM Cortex-M4 Generic User Guide (DUI-0553) Section 2.3, list exception numbers 1–7 with their names and one sentence on what triggers each.

Hint: Table 2-6 in the Cortex-M4 User Guide.

Phase 1.1 GPIO + RCC

These exercises require RM0383. Read the relevant register description before each answer.

E1.1-01 — RCC Clock Enable

Context: This is the first line of every STM32 peripheral init function. Forgetting it causes silent failures.

Question

Write the complete C code to simultaneously enable clocks for GPIOB (bit 1) and GPIOC (bit 2) in RCC_AHB1ENR. Then enable USART1 (bit 4) in RCC_APB2ENR. After each write, add one dummy read to ensure clock propagation. State the register name and bit number for each operation.

Hint: GPIO uses RCC_AHB1ENR. USART1 is on APB2 → RCC_APB2ENR.

E1.1-02 — BSRR vs ODR — Atomic GPIO

Context: BSRR provides atomic set and reset. ODR requires read-modify-write and is not safe when multiple tasks share a GPIO port.

Question

Implement `gpio_write(GPIO_TypeDef *port, uint8_t pin, uint8_t state)` using BSRR only — no read of ODR. Then explain with a concrete 3-step sequence why `ODR ^= (1<<pin)` is unsafe in a FreeRTOS system where two tasks write to different pins on the same port.

Hint: BSRR bits [15:0] set pins. BSRR bits [31:16] reset pins (bit 16+N resets pin N).

E1.1-03 — Alternate Function Configuration

Context: UART, I2C, SPI and Timer pins all require Alternate Function mode. This is more involved than basic output configuration.

Question

Configure PA2 as USART2_TX (AF7). Write the complete sequence: MODER, AFR, OTYPER, OSPEEDR. State which AFR register (AFR[0] or AFR[1]) and which bits. Explain the MODER encoding for AF mode.

Hint: AF mode in MODER = 0b10. Pins 0–7 use AFR[0] (4 bits per pin). AF7 = 0x7.

E1.1-04 — EXTI External Interrupt Setup

Context: You use EXTI for button input and for HC-SR04 echo edge detection in Phase 1.

Question

Configure PA0 as an interrupt source firing on the rising edge. Write the full setup: GPIO config, SYSCFG EXTI mapping, EXTI trigger enable, interrupt unmask, NVIC enable. Write the ISR skeleton including pending-flag clear.

Hint: EXTI line 0 maps to PA0 via SYSCFG_EXTICR1 bits [3:0]=0000. EXTI->PR clears by writing 1.

E1.1-05 — The BSRR Race Condition Proof

Context: This exercise proves you understand atomic operations — a question that appears in embedded internship interviews.

Question

Two FreeRTOS tasks share GPIOA: Task A controls PA5, Task B controls PA6. Both use $\text{ODR} \wedge= (1 \ll \text{pin})$. Describe the exact 5-step sequence that leaves both pins in the wrong state even though each task only modifies its own pin. Then show how BSRR eliminates the race with a one-line fix per task.

Hint: A context switch can happen between the LDR and the STR of the read-modify-write.

Phase 1.2 UART + Ring Buffer

These build your primary debug infrastructure — the tools you rely on for every subsequent phase.

E1.2-01 — BRR Calculation

Context: You must calculate this before writing a single byte of UART driver code.

Question

Calculate the BRR register value for (a) 115200 baud on USART2 with PCLK1=42 MHz and (b) 9600 baud on USART2 same clock. Show full working. State the actual baud error for each.

Hint: $USARTDIV = PCLK/baud$. Integer $\rightarrow$ bits[15:4]. Fraction $\times 16$ rounded $\rightarrow$ bits[3:0].

E1.2-02 — Circular Ring Buffer

Context: This exact buffer runs in your UART ISR and is reused in the Phase 3 protocol parser.

Question

Implement a 256-byte circular ring buffer. Define (a) the struct with head, tail, data[256], (b) rb_push(uint8_t byte) returning 1 on success 0 if full, (c) rb_pop(uint8_t *out) returning 1 on success 0 if empty. Use power-of-2 masking — no modulo, no if-statements for wrap.

Hint: For $N=256$ mask=($N-1$)=0xFF. Full when $(head+1)\&mask==tail$. Empty when $head==tail$.

E1.2-03 — UART ISR Bug Hunt

Context: UART ISR bugs cause intermittent missing bytes that are nearly impossible to reproduce reliably.

Question

Find and fix all three bugs in this ISR:

```
void USART2_IRQHandler(void) {
    uint8_t byte = USART2->DR;
    buf[head] = byte;
    head++;
    if (head >= 256) head = 0;
}
```

Hint: Think: (1) when to read DR, (2) overflow check order, (3) volatile.

E1.2-04 — printf Retargeting via _write

Context: After this change every `printf()` in your project sends output over UART — your primary debug channel.

Question

Write the complete `_write()` syscall override that redirects `printf` to USART2. Include the function signature, the body, and the Makefile flag needed to link the C library correctly.

Hint: newlib calls `_write(fd, ptr, len)` internally for all `printf` output. Link with `-specs=nano.specs`.

E1.2-05 — Systematic UART Fault Diagnosis

Context: Knowing how to diagnose a broken UART systematically saves hours compared to random changes.

Question

Your UART TX produces no output. List 6 possible causes in order of likelihood. For each state the exact diagnostic step — which register to read and what value to expect.

Hint: Work through the chain: RCC → GPIO → UART config → data path.

Phase 1.4 I2C Driver + BMP280 + SSD1306

Have RM0383 Section 27 and the BMP280 datasheet open.

E1.4-01 — I2C CCR and TRISE

Context: These two values must be set correctly before enabling I2C. Wrong values cause timing violations that corrupt data.

Question

Calculate CCR and TRISE for 100 kHz standard mode on I2C1 with PCLK1=42 MHz. Also calculate for 400 kHz fast mode (duty=0, Fm bit set). Show all working for both.

Hint: Sm: $CCR = PCLK1 / (2 \times f)$. Fm duty=0: $CCR = PCLK1 / (3 \times f)$. $TRISE(Sm) = \text{ceil}(1000\text{ns} / t_PCLK) + 1$.
 $TRISE(Fm) = \text{ceil}(300\text{ns} / t_PCLK) + 1$.

E1.4-02 — I2C Write Transaction Trace

Context: Understanding the SR1/SR2 state machine is essential for debugging I2C failures.

Question

Trace the complete I2C state machine for writing byte 0x75 to register 0xF3 of BMP280 at address 0x76. List each step: what you write, what flag you check in SR1 or SR2, and what it means if the flag does not set.

Hint: Sequence: START → check SB → send addr+W → check ADDR → clear ADDR (read SR1 then SR2) → send reg → check TXE → send data → check BTF → STOP.

E1.4-03 — BMP280 Compensation Formula

Context: Raw ADC values from BMP280 are meaningless without the factory calibration compensation.

Question

BMP280 stores calibration trim in registers 0x88–0x9F. Given dig_T1=27504, dig_T2=26435, dig_T3=-1000, raw adc_T=519888, calculate compensated temperature in °C using the integer formula from BMP280 datasheet Section 4.2.3. Show every intermediate variable.

Hint: Two intermediate variables var1 and var2. Result T is in units of 0.01°C.

E1.4-04 — I2C Bus Recovery

Context: The STM32F4 I2C peripheral cannot self-recover from a NACK lockup. This routine is mandatory in production.

Question

Implement `i2c_bus_recovery()` that: (a) disables the I2C peripheral, (b) reconfigures SCL (PB6) and SDA (PB7) as GPIO open-drain outputs, (c) generates 9 SCL pulses with 1ms half-period, (d) sends a manual STOP condition,

(e) reconfigures back to I2C AF mode and calls `i2c_init()`.

Hint: Use `gpio_init()` and `gpio_write()` from your driver. Check SDA state after each pulse.

E1.4-05 — I2C Fault Diagnosis

Context: Systematic diagnosis is faster than random debugging.

Question

Your I2C read of BMP280 `WHO_AM_I` always returns `0xFF`. List 5 causes in order of likelihood. For each state the exact diagnostic step — which register or pin to measure and what to expect.

Hint: `0xFF` means the bus is HIGH — no device acknowledged. Think: pull-ups, address, RCC, GPIO AF, PE bit.

Phase 1.5 SPI Driver

Have RM0383 Section 28 open. SPI exercises are shorter but denser.

E1.5-01 — SPI Mode Matrix

Context: Wrong CPOL/CPHA produces garbage data with no error flag. Always verify the mode with your sensor's datasheet.

Question

Complete this matrix for SPI modes 0–3 (CPOL, CPHA, clock idle state, sampling edge). Which mode does BMP280 use?

Hint: CPOL=0 idles LOW. CPHA=0 samples on first edge.

E1.5-02 — SPI Baud Rate

Context: SPI clock must not exceed the device's maximum. BMP280 maximum SPI clock is 10 MHz.

Question

SPI1 is on APB2 (84 MHz on STM32F411). Calculate the BR[2:0] field needed for the fastest safe clock below 10 MHz. List all 8 possible frequencies.

Hint: SPI baud = $PCLK / 2^{(BR+1)}$. BR is bits [5:3] in SPI_CR1.

E1.5-03 — BMP280 SPI Read Protocol

Context: BMP280 SPI register reads require bit 7 of the address byte to be set to distinguish read from write.

Question

Implement `spi_read_reg(uint8_t reg)` and `spi_write_reg(uint8_t reg, uint8_t data)` with software CS on PA4. Explain why `spi_transfer_byte` sends a dummy 0x00 during the read.

Hint: SPI is full-duplex: every byte sent receives a byte simultaneously.

E1.5-04 — SPI Fault Diagnosis

Context: SPI failures are silent — the transfer completes without any error flag even if the data is wrong.

Question

Your SPI read of BMP280 WHO_AM_I returns 0x00 instead of 0x60. List 5 causes and how to diagnose each. How does the absence of ACK (unlike I2C) change your debugging approach?

Hint: Think about CS timing, MISO/MOSI swap, mode, MISO AF, BMP280 bus selection.

E1.5-05 — SPI vs I2C Decision

Context: Understanding bus selection is a real engineering decision on your PCB.

Question

Fill in the comparison table (wire count, max speed, multi-device, hardware error detection, duplex). For a new sensor sampling at 1 kHz with 12-bit output, which bus would you choose and why?

Hint: I2C: 2 wires, up to 400kHz standard mode. SPI: 4 wires minimum, up to tens of MHz.

Phase 1.6 Timers + HC-SR04 + DHT22

Have RM0383 Section 18 open. Calculator required.

E1.6-01 — Prescaler and ARR Calculation

Context: Every timer-based driver begins with these calculations. Wrong values mean wrong timing with no error.

Question

TIM3 is on APB1. With PCLK1=42 MHz and APB1 prescaler=2 (timer clock = $2 \times \text{PCLK1} = 84 \text{ MHz}$): calculate PSC and ARR for (a) TIM3 ticking at 1 MHz (1 μs per tick) for input capture, (b) TIM2 generating a 1 Hz interrupt (1-second period).

Hint: $f_{\text{tick}} = \text{TIMclk}/(\text{PSC}+1)$. $\text{Period} = (\text{ARR}+1)/f_{\text{tick}}$. Both registers are 16-bit (max 65535).

E1.6-02 — HC-SR04 Distance Calculation

Context: This is the exact calculation in your HC-SR04 driver.

Question

TIM3 runs at 1 MHz (1 $\mu\text{s}/\text{tick}$). Rising edge captured at count=1000, falling edge at count=8762. (a) Echo pulse width? (b) Distance in cm? (c) Maximum detectable distance with 16-bit counter? (d) What happens if echo pulse spans a counter overflow?

Hint: Speed of sound = 343 m/s. Distance = pulse_ μs / 58 (integer approximation).

E1.6-03 — DHT22 Bit Decoding

Context: DHT22 data is encoded entirely in pulse widths — this is the decode logic.

Question

DHT22 timing: '0' bit = 26–28 μs HIGH pulse, '1' bit = 70 μs HIGH pulse (both after 50 μs LOW). Your input capture at 1 MHz gives HIGH pulse widths: [27, 71, 26, 68, 70]. (a) Decode to bits. (b) What threshold divides 0 from 1? (c) What is the total time to receive all 40 bits at worst case?

Hint: Midpoint between 28 and 70 is 49 μs — use 50 as threshold.

E1.6-04 — Why Busy-Wait Breaks Under FreeRTOS

Context: This is why DHT22 must use Timer Input Capture, not delay loops.

Question

Your DHT22 driver uses a busy-wait loop for 50 μs timing between bit reads. It works perfectly without FreeRTOS. With FreeRTOS (SensorAcquisition HIGH, UART_TX MEDIUM) it fails intermittently. Explain the exact failure

mechanism, the observable symptom, and the correct fix.

Hint: Think about what FreeRTOS's SysTick interrupt does mid-loop.

E1.6-05 — Input Capture ISR — HC-SR04

Context: This ISR captures both edges of the echo pulse using a single timer channel.

Question

Write the TIM3 Input Capture ISR that on rising edge stores the count and switches to falling-edge capture, and on falling edge stores the result and signals measurement complete. Handle counter overflow. In FreeRTOS, what replaces the bare volatile flag?

Hint: Toggle CC1P bit in CCER to switch edge polarity between captures.

Phase 2A FreeRTOS Task Architecture

Read FreeRTOS Book Chapters 1–5 before attempting these. All exercises relate directly to your sensor node.

E2A-01 — Stack Size Estimation

Context: Too small = overflow crash. Too large = wasted RAM. You have 128 KB total SRAM.

Question

Estimate the minimum stack size in words for SensorAcquisition task: call depth 4 (task→bmp280_read→i2c_read→i2c_wait), 8 words per frame, 10 local uint32_t variables, one char buf[32]. Add 50% safety margin and FreeRTOS overhead of 64 words. Round up to next multiple of 64.

Hint: Total = (depth × words_per_frame) + locals + overhead. Then ×1.5, round up.

E2A-02 — Priority Inversion — Mutex vs Semaphore

Context: This is a real bug that causes high-priority tasks to miss timing deadlines in production systems.

Question

Three tasks: SensorAcq (HIGH), UART_TX (MEDIUM), SystemHealth (LOW) share the I2C bus via a binary semaphore. Describe the 5-step sequence where SensorAcq is blocked by SystemHealth for an unbounded time. Then explain why a FreeRTOS mutex (not binary semaphore) prevents this.

Hint: Priority inversion: LOW task holds resource, MEDIUM task preempts it, HIGH task waits.

E2A-03 — Queue Design for Sensor Data

Context: Queue too small drops readings. Too large wastes heap. Both have operational consequences.

Question

SensorAcquisition produces at 10 Hz. DataPackager consumes at average 8 Hz (worst case 6.7 Hz at 150 ms). Design SensorDataQueue: item type (with fields), queue length, and what the producer does on overflow. Calculate maximum fill time before overflow at worst-case rates.

Hint: Net accumulation = produce_rate – consume_rate. Queue must absorb a multi-second burst.

E2A-04 — ISR-Safe API — The One Rule

Context: Calling task-context FreeRTOS API from an ISR corrupts the scheduler and causes crashes that appear only hours later.

Question

Your UART RX ISR calls `xQueueSend()` (task API). The system runs for hours then crashes randomly. Explain exactly why it crashes and why it is intermittent. Rewrite the ISR correctly. What does `portYIELD_FROM_ISR()` do?

Hint: `xQueueSend()` may call `taskSuspendAll()` — this is undefined from interrupt context.

E2A-05 — `vTaskDelay` vs `vTaskDelayUntil`

Context: Choosing the wrong delay function causes sensor sampling rate to drift over time, corrupting time-series data.

Question

SensorAcquisition must sample at exactly 10 Hz. Each read takes 15 ms. Show the actual sample rate after 60 seconds using (a) `vTaskDelay(100)` and (b) `vTaskDelayUntil()` with 100 ms period. Calculate total samples for each.

Hint: `vTaskDelay` delays FROM when called. `vTaskDelayUntil` delays TO an absolute tick.

E2A-06 — Deadlock — Detection and Fix

Context: Deadlocks are permanent hangs with no error message. You must recognise the pattern to prevent it.

Question

SensorAcq takes MutexA (I2C) then tries MutexB (UART). CommandHandler takes MutexB then tries MutexA. Describe the 4-step deadlock sequence. How does SystemHealth detect it? What is the one-rule fix that requires no restructuring?

Hint: Deadlock requires circular wait. Eliminating circular wait eliminates deadlock.

Have docs.espressif.com open. These exercises focus on system design decisions.

E2B-01 — MQTT Topic Hierarchy Design

Context: A good topic hierarchy supports multi-device, filtering, and OTA without restructuring.

Question

Design a complete MQTT topic hierarchy for your sensor node supporting: (a) multiple nodes by ID, (b) all 4 sensor types, (c) OTA commands and status, (d) system health, (e) device config. Show publish/subscribe topics for both ESP32 and laptop sides.

Hint: Convention: domain/device_id/category/type. Wildcards: + = one level, # = multi-level.

E2B-02 — MQTT QoS Selection

Context: Wrong QoS wastes bandwidth or risks losing critical messages.

Question

Select QoS 0, 1, or 2 for each and justify: (a) sensor telemetry at 10 Hz, (b) OTA START command, (c) OTA DATA chunk, (d) health ping every 10s, (e) config change.

Hint: QoS 0: fire-and-forget. QoS 1: at-least-once (ACK, may duplicate). QoS 2: exactly-once (4-way handshake).

E2B-03 — WiFi Reconnection Handler

Context: WiFi will disconnect in production. Reconnection must be fully automatic with exponential backoff.

Question

Write the ESP-IDF event handler for WiFi that handles: initial connection, IP assignment, disconnection with exponential backoff (start 1s, double each attempt, cap at 60s), and MQTT reconnection after WiFi is restored.

Hint: WiFi events: WIFI_EVENT_STA_DISCONNECTED. IP event: IP_EVENT_STA_GOT_IP. MQTT reconnect in IP event.

E2B-04 — Partition Table for OTA

Context: The partition table must be correctly designed before OTA is implemented — changing it later requires a physical reflash.

Question

Write the partitions.csv file for a 4MB ESP32 flash that includes NVS, OTA data, PHY calibration, factory app, and two equal OTA slots. Show your size calculations and verify the total does not exceed 4MB (0x400000).

Hint: otadata must be present for OTA to work. OTA slot size limits maximum firmware binary size.

Phase 3 Inter-MCU Protocol Design

These are design and calculation exercises. `protocol_spec.md` should be open alongside.

E3-01 — Frame Field Justification

Context: Every field must earn its bytes — overhead reduces throughput.

Question

Justify the minimum byte width for each field given: max payload=255 bytes, 256 possible command types, rolling sequence for ACK matching over 25-second window at 10 Hz. (a) LEN: why 2 bytes not 1? (b) SEQ: why 1 byte? (c) CMD: why 1 byte? (d) CRC: why 2 bytes not 1?

Hint: Think about the range each field must represent and the window it covers.

E3-02 — Byte Stuffing Encode and Decode

Context: Without byte stuffing, any payload byte equal to SOF (0xAA) or EOF (0x55) corrupts the frame boundary.

Question

Byte stuffing rules: 0xAA→{0xBB,0x8A}, 0x55→{0xBB,0x75}, 0xBB→{0xBB,0x9B}. (a) Encode payload [0x01, 0xAA, 0x03, 0xBB, 0x55]. (b) Decode received sequence [0x42, 0xBB, 0x8A, 0x15, 0xBB, 0x9B]. (c) Worst-case size expansion ratio?

Hint: Scan left to right. Replace each special byte with its 2-byte escape sequence.

E3-03 — CRC-16/CCITT Manual Calculation

Context: Calculating CRC by hand proves you understand the algorithm and validates your implementation.

Question

Calculate CRC-16/CCITT (polynomial 0x1021, init 0xFFFF, no final XOR) for input [0x01, 0x02, 0x03]. Show CRC value after each byte. Verify against crccalc.com.

Hint: For each bit (MSB first): if (CRC MSB) XOR bit == 1 → CRC = (CRC<<1) XOR 0x1021 else CRC = CRC<<1. All 16-bit.

E3-04 — Parser Timeout and Recovery

Context: A state machine without timeout handling freezes your system permanently on any partial frame.

Question

Your parser is in `ST_PAYLOAD` collecting a 20-byte payload. After 10 bytes the UART connection drops. (a) What happens without timeout handling? (b) Implement `parser_timeout_check()` that resets after 500ms of no bytes. (c) What state does it reset to and what fields must be cleared?

Hint: Track `last_byte_time_ms` using `systick_get_ms()`. Reset to `ST_IDLE`, clear all accumulated fields.

Phase 4 Bootloader + OTA Pipeline

Have RM0383 Section 3 (Flash Interface) open. Verify every address calculation twice.

E4-01 — Flash Sector Address Arithmetic

Context: Address errors brick the device. Always verify twice before writing to flash.

Question

STM32F411: Sector 0=0x08000000 (16KB), Sector 4=0x08010000 (64KB), Sectors 5–7=128KB each from 0x08020000. (a) Last byte address of Sector 0? (b) Address of the 256th byte of the application firmware? (c) Firmware is 87420 bytes — does it fit in the application region? How many sectors?

Hint: End = start + size - 1. Firmware byte N (1-indexed) is at app_start + N - 1.

E4-02 — Linker Script Bug — Wrong ORIGIN

Context: This is the most common Phase 4 bug. Getting ORIGIN wrong causes a HardFault on the first instruction after the bootloader jump.

Question

Your bootloader successfully jumps to 0x08010000. The application immediately HardFaults. Application linker script:

```
MEMORY { FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 448K ... }
```

Identify the bug, explain the exact failure mechanism, write the correct linker script.

Hint: Where does the linker think the code lives? Where is the binary actually placed?

E4-03 — Boot Jump Sequence — Find Missing Steps

Context: Each step of the jump sequence has a specific purpose. Missing one causes a specific failure mode.

Question

This boot jump sequence has 3 steps missing or incomplete. Find and fix all 3:

```
void jump(uint32_t app) {  
    __disable_irq();  
    SCB->VTOR = app;  
    void (*reset)(void) = (void (*)(void))(*(uint32_t*)(app+4));  
    reset();  
}
```

Hint: What must happen to SysTick before VTOR change? What barrier after VTOR? What MSP before jump?

E4-04 — OTA Transfer Timeline

Context: Understanding the transfer time helps you set a sensible OTA timeout and choose chunk size.

Question

Firmware = 87,420 bytes. Chunks = 128 bytes. UART = 115,200 baud. (a) How many chunks? (b) Time per chunk frame (7 byte overhead + 128 payload + ~1 byte stuffing)? (c) Total time with no ACKs? (d) Total time with ACK per chunk and 10ms round-trip?

Hint: 1 byte at 115200 baud with 10 bits (start+8data+stop) = $10/115200 \approx 86.8 \mu\text{s}$.

E4-05 — Flash Write with Verification

Context: Flash write failures are silent without explicitly checking the status register after every operation.

Question

Write `flash_write_verified(uint32_t addr, uint32_t data)` that: unlocks, sets x32 parallelism, writes, waits for BSY with timeout, checks all FLASH_SR error bits, reads back to verify, locks. Return a `FlashError` enum.

Hint: FLASH_SR error bits: WRPERR=4, PGAERR=5, PGPERR=6, PGSERR=7. BSY=16.

Solutions

Attempt every exercise before reading this section.

Solutions — Phase 0.1: C Language Hardening

E0.1-01 — Bitwise SET — The RCC Enable Pattern

Solution:

```
uint32_t reg = 0x00000000;
reg |= (1U << 0); // reg = 0x00000001
reg |= (1U << 2) | (1U << 5); // reg = 0x00000025
printf("0x%08X\n", reg); // prints 0x00000025
```

Explanation:

The `|=` operator ORs the mask into the register, leaving all other bits unchanged. `(1U << n)` builds a mask with only bit `n` set. The `U` suffix prevents undefined behaviour on 32-bit signed integers. In your STM32 drivers, `RCC->AHB1ENR |= (1U << 0)` enables GPIOA without touching the other 31 peripheral enable bits.

E0.1-02 — Bitwise CLEAR — The GPIO Configuration Pattern

Solution:

```
uint32_t moder = 0x0000FFFF;
moder &= ~(0x3U << 10); // clear bits [11:10] → 0x00000FFF
moder |= (0x1U << 10); // set to 01 (output) → 0x00000BFF
printf("0x%08X\n", moder); // 0x00000BFF
```

Explanation:

`~(0x3U << 10)` produces `0xFFFFF3FF` — all ones except bits 11 and 10. ANDing clears exactly those two bits. The two-step clear-then-set is the standard safe write for multi-bit fields. If you only OR without clearing, leftover bits from a previous mode corrupt the new setting silently.

E0.1-03 — Bitwise READ — Checking Status Flags

Solution:

```
#define IS_BIT_SET(reg, bit) (((reg) >> (bit)) & 1U)

// Simulate: SR starts at 0, TXE set after 5 iterations
uint32_t USART_SR = 0;
int count = 0;
```

```
while (!IS_BIT_SET(USART_SR, 6)) {
    count++;
    if (count == 5) USART_SR |= (1U << 6); // hardware sets TXE
}

printf("Waited %d iterations\n", count); // 5
```

Explanation:

(reg >> bit) & 1U shifts the target bit to position 0 then masks it, giving exactly 0 or 1. In real drivers USART_SR would be volatile uint32_t* pointing to 0x40004400. The volatile qualifier forces a fresh memory read every loop iteration — without it the compiler reads SR once and the loop never exits in an optimised build.

E0.1-04 — volatile — Why the Compiler Lies Without It

Solution:

```
// BROKEN — no volatile:
struct { uint32_t SR; } *usart = (void*)0x40004400;
while (!(usart->SR & (1<<6))) {}

// Compiler reads SR once, decides the condition is constant,
// and either removes the loop entirely or makes it infinite.

// FIXED:
struct { volatile uint32_t SR; } *usart = (void*)0x40004400;
while (!(usart->SR & (1<<6))) {}

// volatile forces a fresh load instruction on every iteration.
```

Explanation:

The C optimiser assumes that if nothing inside your program writes to usart->SR, it cannot change. volatile tells the compiler: this address can change outside of program flow (the UART hardware changes it). Without volatile, -O2 reads SR once into a register and never reads it again. Every peripheral register pointer in your STM32 drivers must carry volatile — without it everything appears to work at -O0 debug builds and silently fails in release builds.

E0.1-05 — Struct Pointer Casting — The Foundation of Peripheral Drivers

Solution:

```
#define GPIOA_BASE 0x40020000UL

typedef struct {
    volatile uint32_t MODER, OTYPER, OSPEEDR, PUPDR, IDR, ODR, BSRR;
} GPIO_TypeDef;

GPIO_TypeDef *GPIOA = (GPIO_TypeDef *)GPIOA_BASE; // (a)

// (b) Set pin 5 as output (MODER bits [11:10] = 01)
GPIOA->MODER &= ~(0x3U << 10);
GPIOA->MODER |= (0x1U << 10);

// (c) Drive pin 5 HIGH via BSRR (bit 5 in lower half)
GPIOA->BSRR = (1U << 5);
```

Explanation:

The cast `(GPIO_TypeDef *)GPIOA_BASE` tells the compiler to treat the memory at `0x40020000` as having this struct layout. No data is copied — zero cost. The struct fields map to hardware registers at fixed offsets: `MODER` at `+0`, `OTYPER` at `+4`, `BSRR` at `+24`. `BSRR` is write-only: bits `[15:0]` set the corresponding output pin HIGH atomically; bits `[31:16]` clear them. Using `BSRR` instead of `ODR` eliminates the read-modify-write race condition.

E0.1-06 — Function Pointers — The Interrupt Vector Table

Solution:

```
// (a) typedef
typedef void (*VoidHandler)(void);

// (b) forward declarations + dispatch table
void UART_IRQHandler(void);
void I2C_IRQHandler(void);
void SPI_IRQHandler(void);

VoidHandler dispatch[4] = { NULL, UART_IRQHandler, I2C_IRQHandler, SPI_IRQHandler };

// Call entry 2: dispatch[2]() → calls I2C_IRQHandler

// (c) stubs
void UART_IRQHandler(void) { /* clear RXNE, push byte to ring buffer */ }
void I2C_IRQHandler(void) { /* advance I2C state machine */ }
void SPI_IRQHandler(void) { /* read SPI DR, push to buffer */ }
```

Explanation:

`VoidHandler` is a type alias for 'pointer to function taking nothing returning nothing'. The dispatch table stores handler addresses. Calling `dispatch[2]()` dereferences the pointer and jumps to `I2C_IRQHandler`. The Cortex-M startup file works identically: it places handler addresses at fixed flash offsets, and the CPU reads them on reset and on every exception. Understanding this means you can write your own startup file rather than depending on a vendor-supplied one.

E0.1-07 — Challenge — Real STM32 Register Calculation

Solution:

```
uint32_t moder = 0xA8000000; // SWD pins pre-configured

// Pins 0,1,2 → output (01):
moder &= ~(0x3U << 0); moder |= (0x1U << 0); // pin 0 bits [1:0]
moder &= ~(0x3U << 2); moder |= (0x1U << 2); // pin 1 bits [3:2]
moder &= ~(0x3U << 4); moder |= (0x1U << 4); // pin 2 bits [5:4]

// Pin 3 → input (00): clear only, reset value is already 00
moder &= ~(0x3U << 6); // pin 3 bits [7:6]

// Result: 0xA8000015 (verify: printf("0x%08X\n", moder);)
```

Explanation:

Pin `N` uses bits `[2N+1 : 2N]` in `MODER` — shift amount is always `2*N`. Input mode (`00`) only needs clearing because the reset value for user-configurable pins is already `00`. NEVER clear bits `[27:26]`, `[29:28]`, `[31:30]` without restoring them — those control `PA13`, `PA14`, `PA15` which carry SWD signals. Clearing SWD pins disconnects your debugger for the rest of the power cycle.

Solutions — Phase 0.2: Makefile + Toolchain

E0.2-01 — Annotate Every Line of the Minimal Makefile

Solution:

Explanation:

TARGET=firmware: base filename for all output files. MCU: three mandatory flags — cortex-m4 sets the instruction set, mthumb forces Thumb-2 mode (required for all Cortex-M), mfloat-abi enable the hardware FPU. CFLAGS: -O0 disables optimisation so the debugger shows real source lines, -g3 embeds full debug symbols, -Wall enables all warnings. LDFLAGS: -T points to the linker script defining flash/RAM layout, --gc-sections removes dead code (requires -ffunction-sections and -fdata-sections in CFLAGS). all: \$(TARGET).bin makes the .bin the default build target. The .elf rule runs GCC; \$@ is firmware.elf, \$^ is main.c. The .bin rule strips ELF headers — OpenOCD needs raw binary, not ELF.

E0.2-02 — Add size and clean Targets

Solution:

```
.PHONY: all clean size

size: $(TARGET).elf
arm-none-eabi-size $<

# Output: text data bss dec hex filename
# 1234 8 24 1266 4f2 firmware.elf
# Flash = text + data. RAM = data + bss.

clean:
rm -f *.o *.elf *.bin *.map
```

Explanation:

arm-none-eabi-size reads ELF section headers. 'text' = code + read-only constants in flash. 'data' = initialised global variables (stored in flash, copied to RAM at startup). 'bss' = zero-initialised globals (RAM only). Flash usage = text + data. RAM usage = data + bss. When your Phase 4 bootloader binary approaches 32768 bytes (32KB), run 'make size' immediately. Without .PHONY, Make skips 'clean' if a file named 'clean' exists in the directory.

E0.2-03 — The Timestamp Dependency Bug

Solution:

```
# BUG: 'all' is a phony target — no file named 'all' is ever produced.
# Make cannot compare timestamps, so it always runs the rule.

# FIX: use the actual output filename as the target:
firmware.elf: main.c startup.s
arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -o firmware.elf $^

# Now Make checks: is firmware.elf newer than main.c and startup.s?
# If yes → skip. If no → rebuild. Correct incremental behaviour.
```

Explanation:

Make's rebuild logic: rebuild the target only if a prerequisite file is newer than the target file. This only works when the target is an actual file on disk. Writing 'all:' means Make looks for a file called 'all', never finds it, and always rebuilds. The fix is to use the real output filename (firmware.elf) as the target and list actual source files as prerequisites. Make then uses file modification timestamps to decide whether to compile.

E0.2-04 — Compiler Flag Purpose Quiz

Solution:

Explanation:

1. -mfpu=fpv4-sp-d16: enables the F4's hardware FPU for single-precision floats. Without it, all float operations run in slow software emulation — 10–100× slower. 2. -mfloat-abi=hard: pass float arguments in FPU registers (hardware ABI). Must match newlib-nano which also uses hard ABI. Mismatch causes silent wrong values on function calls. 3. -ffunction-sections: places each function in its own ELF section (.text.functionname). Required for -gc-sections to know which sections are unused. 4. -Wl,-gc-sections: the linker removes ELF sections unreachable from the entry point. Without it, every function in every .c file (including unused library code) bloats the binary. 5. -specs=nano.specs: links against newlib-nano — a minimal C library optimised for embedded. Provides printf, malloc, and syscall stubs. Without it, full newlib adds ~100KB of printf formatting code you don't need.

E0.2-05 — Linker Script Anatomy

Solution:

Explanation:

MEMORY block defines the physical memory regions available. FLASH at 0x08000000 is the STM32's internal flash — code goes here. RAM at 0x20000000 is SRAM — variables go here. (rx) = readable + executable, (rwx) = all three. SECTIONS maps C objects to regions: .text contains compiled machine code → placed in FLASH. .data contains initialised global variables — stored in FLASH (AT > FLASH) but executed from RAM (> RAM) because the CPU cannot write to flash at runtime; the startup file copies .data from flash to RAM before main(). .bss contains zero-initialised globals — only RAM needed, startup fills them with zeros. For Phase 4 the bootloader linker script uses LENGTH=32K for FLASH, the application linker script uses ORIGIN=0x08010000 LENGTH=448K.

Solutions — Phase 0.3: Datasheet Navigation

E0.3-01 — Memory Map — Peripheral Addresses

Solution:

```
GPIOA: 0x40020000 (AHB1 — max 84 MHz on F411)
USART1: 0x40011000 (APB2 — max 84 MHz on F411)
I2C1: 0x40005400 (APB1 — max 42 MHz on F411)
SPI1: 0x40013000 (APB2 — max 84 MHz on F411)
TIM2: 0x40000000 (APB1 — max 42 MHz on F411)
```

Explanation:

Bus matters for baud-rate calculations. USART1 on APB2 uses 84 MHz as its peripheral clock — $BRR = 84,000,000 / \text{baud}$. USART2 on APB1 uses 42 MHz — $BRR = 42,000,000 / \text{baud}$. Using the wrong clock frequency produces a baud rate error that causes garbled UART output with no error message. The RCC enable registers are also split by bus: RCC_AHB1ENR for GPIO, RCC_APB1ENR for I2C/TIM2, RCC_APB2ENR for USART1/SPI1.

E0.3-02 — UART BRR Register Calculation

Solution:

```
USARTDIV = 42,000,000 / 115,200 = 364.583

Integer part: 364 = 0x16C → bits [15:4]
Fraction part: 0.583 × 16 = 9.33 → round to 9 = 0x9 → bits [3:0]

BRR = (364 << 4) | 9 = 0x16C9

USART2->BRR = 0x16C9;

// Verify: actual = 42,000,000 / (16 × 364.5625) = 115,207 Hz
// Error = (115,207 - 115,200) / 115,200 = 0.006% (well within ±2% spec)
```

Explanation:

BRR is a fixed-point register: 12-bit integer + 4-bit sixteenths fraction. Multiply the fractional part by 16 and round to get the BRR fraction field. Always verify by computing reverse: $\text{actual} = \text{PCLK} / (16 \times (\text{integer} + \text{fraction}/16))$. For USART1 on APB2 use 84,000,000 in the formula. A baud rate error below 2% is acceptable per STM32 electrical specifications.

E0.3-03 — GPIO Configuration Bit Fields

Solution:

```
// PA5 = pin 5 of GPIOA

GPIOA->MODER bits [11:10] = 0b01 output mode
→ GPIOA->MODER &= ~(0x3U<<10); GPIOA->MODER |= (0x1U<<10);

GPIOA->OTYPER bit [5] = 0 push-pull (not open-drain)
→ GPIOA->OTYPER &= ~(1U<<5);
```



```
GPIOA->OSPEEDR bits [11:10] = 0b01 medium speed (25 MHz)
→ GPIOA->OSPEEDR &= ~(0x3U<<10); GPIOA->OSPEEDR |= (0x1U<<10);

GPIOA->PUPDR bits [11:10] = 0b00 no pull
→ GPIOA->PUPDR &= ~(0x3U<<10);
```

Explanation:

Each pin in MODER, OSPEEDR, and PUPDR uses 2 bits at position $2 \times N$. Each pin in OTYPER uses 1 bit at position N . So pin 5 uses bits [11:10] in the 2-bit registers and bit [5] in OTYPER. Speed controls slew rate — higher speed increases EMI. For LED driving, medium speed is correct. Never set speed higher than the signal frequency requires.

E0.3-04 — I2C CCR Calculation

Solution:

```
CCR = 42,000,000 / (2 × 100,000) = 210 = 0xD2

TRISE = floor(1000 ns / (1/42 MHz)) + 1
= floor(1000 × 42 / 1000) + 1
= 42 + 1 = 43 = 0x2B

I2C1->CCR = 210;
I2C1->TRISE = 43;
```

Explanation:

CCR divides PCLK1 to produce the SCL clock. The factor of 2 gives 50% duty cycle for standard mode. TRISE limits the rise-time detector — it tells the peripheral how many PCLK1 cycles correspond to the maximum allowed SDA/SCL rise time. Setting TRISE incorrectly causes the peripheral to sample data before signals are stable, producing intermittent corrupt reads. Both CCR and TRISE must be set before enabling I2C (setting PE bit in CR1).

E0.3-05 — Cortex-M4 Exception Types

Solution:

Explanation:

1=Reset: power-on or external NRST pin. 2=NMI: Non-Maskable Interrupt, cannot be disabled, used for critical hardware faults. 3=HardFault: escalation from any other fault when the handler is disabled or another fault occurs during fault handling. 4=MemManage: MPU violation — accessing a protected or forbidden memory region. 5=BusFault: error on instruction fetch or data read/write — typically accessing an invalid address. 6=UsageFault: undefined instruction, invalid EXC_RETURN, unaligned access (if enabled), divide by zero (if enabled). 7=Reserved. In Phase 1 you will primarily encounter HardFault (3) from null-pointer dereferences and unaligned accesses. Your HardFault handler prints PC and LR so you can identify the exact line that crashed.

Solutions — Phase 1.1: GPIO + RCC

E1.1-01 — RCC Clock Enable

Solution:

```
// Enable GPIOB (bit 1) and GPIOC (bit 2) simultaneously:
RCC->AHB1ENR |= (1U << 1) | (1U << 2);
volatile uint32_t dummy = RCC->AHB1ENR; (void)dummy; // propagation delay

// Enable USART1 (bit 4):
RCC->APB2ENR |= (1U << 4);
dummy = RCC->APB2ENR; (void)dummy;
```

Explanation:

Setting two bits with a single `|=` is atomic on Cortex-M (one read-modify-write instruction). The dummy read after enabling ensures the clock is actually routed to the peripheral before the first register access — on some STM32 variants, the first access to a peripheral whose clock was just enabled can fault without this delay. bit positions come from RM0383 Table 27 (AHB1ENR) and Table 31 (APB2ENR). Always verify bit numbers from the RM — they vary between STM32 families.

E1.1-02 — BSRR vs ODR — Atomic GPIO

Solution:

```
void gpio_write(GPIO_TypeDef *port, uint8_t pin, uint8_t state) {
    if (state)
        port->BSRR = (1U << pin); // atomic SET
    else
        port->BSRR = (1U << (pin + 16)); // atomic RESET
}

// ODR race – Task A controls PA5, Task B controls PA6:
// 1. Task A reads ODR = 0x0040 (PA6 is HIGH)
// 2. Context switch to Task B
// 3. Task B reads ODR = 0x0040, clears bit 6 → ODR = 0x0000, writes back (PA6 LOW)
// 4. Context switch back to Task A
// 5. Task A uses stale value 0x0040, ORs bit 5 → 0x0060, writes back → PA6 HIGH again!
// Task B's write was silently overwritten.
```

Explanation:

BSRR is write-only: each 32-bit write to BSRR is a single STR instruction, atomic at the hardware level. No read means no race condition. ODR read-modify-write needs three instructions (LDR, ORR, STR) — a context switch between any two corrupts the operation. In FreeRTOS with multiple tasks driving different pins on the same port, BSRR is mandatory. A mutex around every ODR access is an alternative but adds latency; BSRR is both faster and correct.

E1.1-03 — Alternate Function Configuration

Solution:

```
// PA2 → USART2_TX = AF7

// 1. MODER[5:4] = 0b10 (alternate function)
GPIOA->MODER &= ~(0x3U << 4);
GPIOA->MODER |= (0x2U << 4);

// 2. AFR[0] bits [11:8] = 0x7 (AF7, pin 2, 4 bits each)
GPIOA->AFR[0] &= ~(0xFU << 8);
GPIOA->AFR[0] |= (0x7U << 8);

// 3. Push-pull output type
GPIOA->OTYPER &= ~(1U << 2);

// 4. Medium speed
GPIOA->OSPEEDR &= ~(0x3U << 4);
GPIOA->OSPEEDR |= (0x1U << 4);

// 5. No pull
GPIOA->PUPDR &= ~(0x3U << 4);
```

Explanation:

MODER=10 connects the pin to the peripheral's alternate function output, not to ODR. AFR[0] is AFRL (alternate function low register) covering pins 0–7; AFR[1] is AFRH for pins 8–15. Each pin uses 4 bits (16 possible AFs). AF number assignment is device-specific — always look it up in the STM32F411 datasheet Alternate Functions table, not in RM0383. The RM describes the registers; the datasheet says which AF number maps to which peripheral on each pin.

E1.1-04 — EXTI External Interrupt Setup

Solution:

```
// Clock enables
RCC->APB2ENR |= (1U << 14); // SYSCFGEN
RCC->AHB1ENR |= (1U << 0); // GPIOAEN

// PA0 as input with pull-up (button active-low)
GPIOA->MODER &= ~(0x3U << 0); // input = 00
GPIOA->PUPDR &= ~(0x3U << 0);
GPIOA->PUPDR |= (0x1U << 0); // pull-up

// Map EXTI0 to GPIOA (SYSCFG_EXTICR1 bits [3:0] = 0000)
SYSCFG->EXTICR[0] &= ~(0xFU << 0);

// Rising edge trigger, disable falling
EXTI->RTSR |= (1U << 0);
EXTI->FTSR &= ~(1U << 0);

// Unmask EXTI line 0
EXTI->IMR |= (1U << 0);

// Enable in NVIC
NVIC_SetPriority(EXTI0_IRQn, 5);
NVIC_EnableIRQ(EXTI0_IRQn);
```

```
// ISR:

void EXTI0_IRQHandler(void) {
    if (EXTI->PR & (1U << 0)) {
        EXTI->PR = (1U << 0); // clear pending – write 1 to clear
        // handle button press
    }
}
```

Explanation:

EXTI needs SYSCFG to select which GPIO port drives each EXTI line — up to one pin per number across all ports can trigger the same EXTI line. SYSCFG_EXTICR[0] bits [3:0]=0000 selects GPIOA for EXTI0. The pending register (EXTI->PR) is unusual: writing 1 clears the flag, writing 0 has no effect. NVIC priority must be set before enabling. In FreeRTOS, NVIC priority must be $\geq$ configMAX_SYSCALL_INTERRUPT_PRIORITY (usually 5) to use FromISR APIs safely.

E1.1-05 — The BSRR Race Condition Proof

Solution:

```
// THE BUG (ODR):
// State: PA5=LOW, PA6=HIGH → ODR = 0x0040

// 1. Task A: LDR r0, [ODR] → r0 = 0x0040
// 2. ** CONTEXT SWITCH TO TASK B **
// 3. Task B: LDR r1, [ODR] → r1 = 0x0040
// Task B: ORR r1, #(1<<6) → r1 = 0x0040 (bit6 already set, XOR clears it: 0x0000)
// Wait – Task B wants to TOGGLE PA6: r1 = 0x0040 ^ 0x0040 = 0x0000
// Task B: STR r1, [ODR] → ODR = 0x0000 (PA6 now LOW)
// 4. ** CONTEXT SWITCH BACK TO TASK A **
// 5. Task A: ORR r0, #(1<<5) → r0 = 0x0060 (using STALE 0x0040)
// Task A: STR r0, [ODR] → ODR = 0x0060 (PA6 HIGH again – Task B's write lost!)

// THE FIX (BSRR – one atomic STR, no read):
// Task A toggle PA5: use XOR of current state tracked in software
// Task A set PA5: GPIOA->BSRR = (1U << 5);
// Task A clr PA5: GPIOA->BSRR = (1U << 21);
// Task B set PA6: GPIOA->BSRR = (1U << 6); – independent, no conflict
```

Explanation:

BSRR resolves races because each write is a single STR instruction — the CPU cannot be interrupted mid-write (STR is atomic at the instruction level). ODR read-modify-write requires three instructions minimum. Any context switch between LDR and STR corrupts the operation. This is not a theoretical concern — it causes real bugs in production FreeRTOS systems. The only safe use of ODR in a multi-task system is when all tasks that touch a given port are protected by a single mutex, but BSRR removes the need entirely.

Solutions — Phase 1.2: UART + Ring Buffer

E1.2-01 — BRR Calculation

Solution:

```
// (a) 115200 baud, PCLK1=42 MHz:
USARTDIV = 42,000,000 / 115,200 = 364.583
Integer = 364 = 0x16C
Fraction = 0.583 × 16 = 9.33 → 9 = 0x9
BRR = 0x16C9
Actual = 42,000,000 / (16 × 364.5625) = 115,207 Hz error = 0.006%

// (b) 9600 baud:
USARTDIV = 42,000,000 / 9,600 = 4375.0 (exact)
Integer = 4375 = 0x1117
Fraction = 0 × 16 = 0
BRR = 0x11170
Actual = 9600 Hz error = 0.000%
```

Explanation:

BRR is a fixed-point register. Integer part in bits[15:4] (12-bit), fraction in bits[3:0] (4-bit, represents sixteenths). For 115200 the error is tiny (0.006%) because 42 MHz is close to a multiple. For 9600 it is exact. Errors below 2% are within spec. If you ever see garbled output, the first thing to check is the BRR value and whether you used the correct peripheral clock frequency.

E1.2-02 — Circular Ring Buffer

Solution:

```
#define RB_SIZE 256
#define RB_MASK (RB_SIZE - 1) // 0xFF

typedef struct {
    volatile uint8_t data[RB_SIZE];
    volatile uint32_t head; // ISR writes here
    volatile uint32_t tail; // task reads here
} RingBuf;

static RingBuf rb;

uint8_t rb_push(uint8_t byte) {
    uint32_t next = (rb.head + 1) & RB_MASK;
    if (next == rb.tail) return 0; // full — check BEFORE write
    rb.data[rb.head] = byte;
    rb.head = next;
    return 1;
}

uint8_t rb_pop(uint8_t *out) {
    if (rb.head == rb.tail) return 0; // empty
```

```

*out = rb.data[rb.tail];

rb.tail = (rb.tail + 1) & RB_MASK;

return 1;

}

```

Explanation:

Power-of-2 masking replaces modulo with one AND instruction — faster on a CPU with no divide unit. The full check uses one wasted slot: capacity is $RB_SIZE-1=255$ to distinguish full ($next_head==tail$) from empty ($head==tail$). Both head and tail are volatile: head is written by the ISR, tail by the task — without volatile the compiler may cache values in registers and the two contexts see stale data. This single-producer (ISR) single-consumer (task) pattern is safe without a mutex because each side only modifies its own pointer.

E1.2-03 — UART ISR Bug Hunt

Solution:

```

// BUG 1: No RXNE check. ISR can fire for ORE (overflow) or other flags.
// Reading DR then may return garbage and clear a valid RXNE.
// BUG 2: Overflow check AFTER the write. When buffer is full, byte is
// written to buf[head] even though there is no room.
// BUG 3: head is not volatile – compiler may cache it in a register.
// FIXED:
void USART2_IRQHandler(void) {
    if (USART2->SR & (1U << 5)) { // Bug 1: check RXNE first
        uint8_t byte = (uint8_t)USART2->DR; // reading DR clears RXNE
        uint32_t next = (head + 1) & 0xFF;
        if (next != tail) { // Bug 2: check BEFORE writing
            buf[head] = byte;
            head = next;
        }
        // If full: drop byte (acceptable – add overflow counter)
    }
    // Bug 3: declare head as: volatile uint32_t head;
}

```

Explanation:

RXNE (receive not empty) bit 5 of USART_SR means a received byte is waiting in DR. Without checking it, the ISR runs for ORE and other events and reads garbage from DR. Reading DR automatically clears RXNE. The overflow check must happen before the write — checking after means you write to `buf[head]` when $head+1==tail$, corrupting the byte at the tail position. volatile on head is mandatory because it is written by the ISR context and read by the main task context.

E1.2-04 — printf Retargeting via _write

Solution:

```

// In syscalls.c or main.c:

#include

```

```
int _write(int fd, char *ptr, int len) {
    // fd 1=stdout, 2=stderr – both go to UART
    uart_write_buf((const uint8_t *)ptr, (size_t)len);
    return len; // must return bytes written or printf internal state breaks
}

// Makefile LDFLAGS already needs:
// -specs=nano.specs – newlib-nano + stub syscalls
// This provides _sbrk, _close, _fstat etc. as stubs.
// Your _write overrides the stub for stdout.

// Test immediately after:
printf("Temp: %d.%02d C\r\n", 25, 8); // should appear in minicom
```

Explanation:

newlib implements printf by calling _write() at the bottom of the output chain. Providing your own _write redirects all printf output to UART. The function must return len (bytes written) — returning anything else breaks printf's internal buffer tracking. --specs=nano.specs links newlib-nano (smaller than full newlib) and provides default no-op stubs for all other syscalls (_read, _sbrk, _close, etc.) that your bare-metal program doesn't need. This is a one-time 5-line change that enables formatted output for the entire project.

E1.2-05 — Systematic UART Fault Diagnosis

Solution:

Explanation:

1. RCC clock not enabled (most common). Check: RCC->APB1ENR bit 17 (USART2EN) must be 1. 2. GPIO not in AF mode. Check: GPIOA->MODER bits [5:4] must be 0b10 (10 = AF). 3. Wrong alternate function number. Check: GPIOA->AFR[0] bits [11:8] must be 0x7 (AF7 = USART2_TX on PA2). 4. Wrong BRR value. Check: USART2->BRR must equal your calculated value (0x16C9 for 115200 at 42 MHz). 5. USART not enabled or transmitter disabled. Check: USART2->CR1 bit 13 (UE) and bit 3 (TE) must both be 1. 6. PA2 floating or wrong pin. Check: with multimeter at idle USART2 TX should read 3.3V (UART idles HIGH). If 0V the GPIO is not configured correctly.

Solutions — Phase 1.4: I2C Driver + BMP280 + SSD1306

E1.4-01 — I2C CCR and TRISE

Solution:

```
// Standard mode 100 kHz, PCLK1=42 MHz:
CCR = 42,000,000 / (2 × 100,000) = 210
TRISE = floor(1000 × 42/1000) + 1 = 43
I2C1->CCR = 210;
I2C1->TRISE = 43;

// Fast mode 400 kHz, duty=0:
CCR = 42,000,000 / (3 × 400,000) = 35
TRISE = floor(300 × 42/1000) + 1 = 13
I2C1->CCR = (1U<<15) | 35; // Fm bit + value
I2C1->TRISE = 13;
```

Explanation:

In standard mode SCL has 50% duty so each half-period = $1/(2 \times f_{SCL})$. CCR counts PCLK1 cycles per half-period. In fast mode duty=0 the low period is 2× the high period so the divisor becomes 3. TRISE is the number of PCLK1 cycles in the maximum allowed rise time: 1000 ns (standard) or 300 ns (fast). Always set TRISE and CCR before enabling the peripheral (PE bit). Setting them after causes the first transaction to use wrong timing.

E1.4-02 — I2C Write Transaction Trace

Solution:

```
I2C1->CR1 |= (1U<<8); // send START
while (!(I2C1->SR1 & (1U<<0))); // wait SB=1 (start generated)

I2C1->DR = (0x76 << 1) | 0; // send addr + write bit
while (!(I2C1->SR1 & (1U<<1))); // wait ADDR=1 (address sent, ACK received)
(void)I2C1->SR1; (void)I2C1->SR2; // clear ADDR by reading SR1 then SR2

I2C1->DR = 0xF3; // send register address
while (!(I2C1->SR1 & (1U<<7))); // wait TXE=1 (TX buffer empty)

I2C1->DR = 0x75; // send data
while (!(I2C1->SR1 & (1U<<2))); // wait BTF=1 (byte transfer finished)
// BTF means both DR and shift register empty – last bit was clocked out

I2C1->CR1 |= (1U<<9); // send STOP
```

Explanation:

ADDR is cleared by reading SR1 followed immediately by SR2 — reading SR1 alone does not clear it. This two-read sequence is documented in RM0383 and is mandatory. BTF (Byte Transfer Finished) means the shift register and data register are both empty — sending STOP before BTF can cut off the last bit of the final byte. Add a 5ms timeout using `systick_get_ms()` to every while loop — if the bus is locked the loop would otherwise spin forever.

E1.4-03 — BMP280 Compensation Formula

Solution:

```
int32_t adc_T = 519888;
int32_t dig_T1=27504, dig_T2=26435, dig_T3=-1000;

int32_t var1 = (((adc_T>>3) - ((int32_t)dig_T1<<1)) * dig_T2) >> 11;
// = ((64986 - 55008) * 26435) >> 11 = (9978 * 26435) >> 11 = 128,754

int32_t var2 = (((((adc_T>>4) - (int32_t)dig_T1) *
((adc_T>>4) - (int32_t)dig_T1)) >> 12) *
(int32_t)dig_T3 >> 14;
// = ((32493-27504)^2 >> 12) * (-1000) >> 14 = -370

int32_t t_fine = var1 + var2; // = 128,384
int32_t T = (t_fine * 5 + 128) >> 8;
// T = 2508 → temperature = T/100 = 25.08 °C
```

Explanation:

The compensation formula uses a polynomial correction with factory-calibrated coefficients stored in the sensor. `t_fine` is an intermediate value shared with the pressure compensation formula — you must compute temperature first to get `t_fine`, then use it in pressure compensation. Result `T` is in 0.01°C units so divide by 100 to get Celsius. Read all 6 trim bytes during init and store them in a struct — they are written once at the factory and do not change at runtime.

E1.4-04 — I2C Bus Recovery

Solution:

```
void i2c_bus_recovery(void) {
    I2C1->CR1 &= ~(1U << 0); // disable I2C (PE=0)

    // Reconfigure PB6 (SCL) and PB7 (SDA) as open-drain GPIO outputs
    gpio_init(GPIOB, 6, GPIO_MODE_OUTPUT, GPIO_OTYPE_OD, GPIO_SPEED_LOW, GPIO_PULL_NONE);
    gpio_init(GPIOB, 7, GPIO_MODE_INPUT, GPIO_OTYPE_OD, GPIO_SPEED_LOW, GPIO_PULL_UP);
    gpio_write(GPIOB, 7, 1); // SDA high

    for (int i = 0; i < 9; i++) {
        gpio_write(GPIOB, 6, 0); systick_delay_ms(1); // SCL low
        gpio_write(GPIOB, 6, 1); systick_delay_ms(1); // SCL high
        if (gpio_read(GPIOB, 7)) break; // SDA released – slave freed
    }

    // Manual STOP: SDA goes high while SCL is high
    gpio_init(GPIOB, 7, GPIO_MODE_OUTPUT, GPIO_OTYPE_OD, GPIO_SPEED_LOW, GPIO_PULL_NONE);
    gpio_write(GPIOB, 7, 0); systick_delay_ms(1);
    gpio_write(GPIOB, 6, 1); systick_delay_ms(1);
    gpio_write(GPIOB, 7, 1); systick_delay_ms(1); // STOP

    // Restore I2C AF mode
    gpio_init(GPIOB, 6, GPIO_MODE_AF, GPIO_OTYPE_OD, GPIO_SPEED_MEDIUM, GPIO_PULL_UP);
    gpio_init(GPIOB, 7, GPIO_MODE_AF, GPIO_OTYPE_OD, GPIO_SPEED_MEDIUM, GPIO_PULL_UP);
}
```

```
GPIOB->AFR[0] |= (4U << 24) | (4U << 28); // AF4 = I2C1
i2c_init();
}
```

Explanation:

STM32F4 I2C errata: after a NACK, SDA can be stuck LOW because the slave is waiting for more clock pulses to complete its current byte. Nine clock pulses force the slave to finish the byte and release SDA. The STOP condition tells the slave the transaction ended. Re-initialise the I2C peripheral from scratch after recovery — its internal state is corrupted. Call this function from inside your i2c timeout handler whenever any polling loop times out with the bus appearing stuck.

E1.4-05 — I2C Fault Diagnosis

Solution:

Explanation:

1. Wrong I2C address. BMP280 address is 0x76 when SDO=LOW, 0x77 when SDO=HIGH. If SDO is floating you get an unpredictable address. Diagnostic: try both 0x76 and 0x77. 2. Missing pull-up resistors. I2C open-drain lines cannot pull HIGH without external pull-ups. Diagnostic: measure SDA and SCL with multimeter at idle — both should read 3.3V. If 0V or floating, pull-ups are missing. 3. RCC clock not enabled. Diagnostic: read RCC->APB1ENR bit 21 (I2C1EN) — must be 1. 4. GPIO not in AF mode. Diagnostic: read GPIOB->MODER bits [13:12] and [15:14] — both must be 0b10. 5. I2C peripheral not enabled. Diagnostic: read I2C1->CR1 bit 0 (PE) — must be 1. 0xFF means the master sent the address byte but received no ACK from any device.

Solutions — Phase 1.5: SPI Driver

E1.5-01 — SPI Mode Matrix

Solution:

```
Mode 0: CPOL=0 CPHA=0 idle=LOW sample=Rising (1st edge)
Mode 1: CPOL=0 CPHA=1 idle=LOW sample=Falling (2nd edge)
Mode 2: CPOL=1 CPHA=0 idle=HIGH sample=Falling (1st edge)
Mode 3: CPOL=1 CPHA=1 idle=HIGH sample=Rising (2nd edge)

BMP280 supports Mode 0 and Mode 3 (same relative edge timing).
Default: Mode 0. SPI1->CR1 CPOL=0 CPHA=0 → both bits [1:0] = 0.
```

Explanation:

CPOL sets the clock polarity at idle. CPHA selects the edge for data sampling (first or second transition). BMP280 datasheet Section 6.3 confirms modes 0 and 3. Always check the target device's datasheet — do not assume Mode 0. Getting CPOL or CPHA wrong causes data to be sampled half a clock cycle too early or late, producing a bit-shifted wrong value with no error indication.

E1.5-02 — SPI Baud Rate

Solution:

```
PCLK2 = 84 MHz
BR=0: 84/2 = 42.0 MHz too fast
BR=1: 84/4 = 21.0 MHz too fast
BR=2: 84/8 = 10.5 MHz too fast (>10 MHz limit)
BR=3: 84/16 = 5.25 MHz ✓ fastest safe
BR=4: 84/32 = 2.625 MHz
BR=5: 84/64 = 1.3125 MHz
BR=6: 84/128 = 0.656 MHz
BR=7: 84/256 = 0.328 MHz

SPI1->CR1 |= (3U << 3); // BR=3 → divide by 16
```

Explanation:

BR=2 gives 10.5 MHz which exceeds BMP280's 10 MHz limit — do not use it. BR=3 at 5.25 MHz gives a safe margin. The baud rate formula is $PCLK / 2^{(BR+1)}$. SPI1 on APB2 is faster than SPI2/SPI3 on APB1 — use SPI1 when you need higher throughput. For BMP280, BR=3 is fine. If you later add a display controller that supports 20+ MHz you can increase BR accordingly.

E1.5-03 — BMP280 SPI Read Protocol

Solution:

```
void cs_low(void) { GPIOA->BSRR = (1U<<(4+16)); }
void cs_high(void) { GPIOA->BSRR = (1U<<4); }

uint8_t spi_transfer_byte(uint8_t tx) {
```

```

while (!(SPI1->SR & (1U<<1))); // wait TXE
SPI1->DR = tx;
while (!(SPI1->SR & (1U<<0))); // wait RXNE
return (uint8_t)SPI1->DR;
}

uint8_t spi_read_reg(uint8_t reg) {
cs_low();
spi_transfer_byte(reg | 0x80); // bit7=1 = read
uint8_t val = spi_transfer_byte(0x00); // dummy out, data in
cs_high();
return val;
}

void spi_write_reg(uint8_t reg, uint8_t data) {
cs_low();
spi_transfer_byte(reg & 0x7F); // bit7=0 = write
spi_transfer_byte(data);
cs_high();
}

```

Explanation:

SPI is full-duplex: the master clocks out one byte and simultaneously clocks in one byte. During the first transfer (address byte) the slave is still processing the command and the received byte is garbage — discard it. During the second transfer (dummy 0x00 out) the slave clocks out the register value — this is the byte you read. CS must remain LOW for the entire multi-byte transaction. CS high between bytes would restart the transaction from the beginning.

E1.5-04 — SPI Fault Diagnosis

Solution:

Explanation:

1. MISO not connected or MISO/MOSI swapped. Diagnostic: probe MISO line with multimeter — if stuck at 0V, MISO is not connected or shorted to GND. 2. CS never asserted. Diagnostic: probe CS pin — should drop to 0V during transfer. If it stays at 3.3V, your CS gpio_write(0) is not working. 3. BMP280 in I2C mode. The BMP280 selects its bus on power-on based on CS state — if CS is floating or HIGH at power-on it initialises in I2C mode and ignores SPI commands. Solution: hold CS LOW before power-on. 4. Wrong SPI mode. Diagnostic: logic analyser showing data being sampled on the wrong edge. 5. MISO GPIO not in AF mode. Diagnostic: read GPIOA->MODER for the MISO pin — must be 0b10. Key difference from I2C: SPI has no ACK/NAK so you cannot tell from software whether the device responded at all. The logic analyser is essential for SPI debugging — you must see the MISO line to know if data is coming back.

E1.5-05 — SPI vs I2C Decision

Solution:

```

Wire count: I2C=2 (SDA+SCL) SPI=4+ (MOSI/MISO/SCK/CS per device)
Max speed: I2C=400kHz (Fm) / 3.4MHz SPI=10-50+ MHz (device limited)
Multi-device:I2C=address-based, 2 wires SPI=one CS pin per device

```

```
Error detect:I2C=ACK/NAK per byte SPI=none (add your own CRC if needed)
```

```
Duplex: I2C=half-duplex only SPI=full-duplex
```

```
Decision for 1kHz / 12-bit sensor:
```

```
Data rate = 1000 × 16 = 16 kbps → either bus is fast enough.
```

```
Choose I2C: simpler wiring, fewer GPIO pins, ACK confirmation per byte.
```

```
Choose SPI only if: speed >400kHz, need full-duplex, or I2C bus is full.
```

Explanation:

At 1kHz with 12-bit data the bandwidth requirement is tiny — 16kbps vs I2C's 400kbps capability. The engineering trade-off favours I2C: 2 fewer wires, built-in error detection (ACK/NAK), multiple devices on one bus. Use SPI when: the sensor requires it, data rates exceed 400kHz, low-latency continuous streaming is needed (IMU at 8kHz for example), or when you need simultaneous send/receive (audio codec). Your BMP280 and SSD1306 are both I2C-capable — use I2C for those and reserve SPI for devices that genuinely require it.

Solutions — Phase 1.6: Timers + HC-SR04 + DHT22

E1.6-01 — Prescaler and ARR Calculation

Solution:

```
// Timer clock = 84 MHz (2 × PCLK1 when APB1 prescaler ≠ 1)

// (a) TIM3 at 1 MHz:
// PSC+1 = 84,000,000 / 1,000,000 = 84 → PSC = 83
TIM3->PSC = 83;
TIM3->ARR = 0xFFFF; // free-running counter for input capture

// (b) TIM2 at 1 Hz (period = 1 second):
// 1 MHz tick: ARR+1 = 1,000,000 → ARR = 999,999 > 65535 (won't fit!)
// Solution: use 10 kHz tick → PSC = 8399, ARR = 9999
// 84,000,000 / 8400 = 10,000 Hz. 10,000 × (9999+1) = 100,000,000 ns = 1 s
TIM2->PSC = 8399; // 84 MHz / 8400 = 10 kHz
TIM2->ARR = 9999; // 10,000 counts at 10 kHz = 1 second
TIM2->DIER |= (1U << 0); // enable update interrupt
```

Explanation:

The timer clock on STM32F4 is doubled when the APB prescaler is not 1 — documented in RM0383 Section 6.2. PSC and ARR are zero-indexed: actual division is (PSC+1), actual period is (ARR+1) counts. A 16-bit ARR limits maximum count to 65535 — for long periods you must increase PSC. Verification formula: $\text{period}_s = (\text{PSC}+1) \times (\text{ARR}+1) / \text{TIMclk}$. For TIM2: $8400 \times 10000 / 84,000,000 = 1.0$ s exactly.

E1.6-02 — HC-SR04 Distance Calculation

Solution:

```
// (a) Pulse width:
width_us = 8762 - 1000 = 7762 μs

// (b) Distance:
distance_cm = 7762 / 58 = 133 cm
// Exact: (7762 × 0.0343) / 2 = 133.1 cm

// (c) Maximum with 65535 μs range:
max_cm = 65535 / 58 = 1130 cm = 11.3 m
// HC-SR04 datasheet max = 400 cm — sensor limits before counter does

// (d) Counter overflow handling:
// If falling_capture < rising_capture, counter wrapped.
// Corrected width = (falling + 65536) - rising
if (falling < rising)
width_us = (falling + 65536) - rising;
else
width_us = falling - rising;
```

Explanation:

The /58 approximation: sound travels to the object and back, so one-way distance = $\text{pulse_}\mu\text{s} \times 343/2 \text{ m}/\mu\text{s} = \text{pulse_}\mu\text{s} \times 0.01715 \text{ cm}/\mu\text{s} = \text{pulse_}\mu\text{s} / 58.3$. Integer division by 58 gives $\pm 1 \text{ cm}$ error — acceptable for HC-SR04's stated $\pm 3\text{mm}$ accuracy. Add a 40 ms timeout: if no falling edge arrives within 40,000 μs (HC-SR04 max spec), abort and return 0 (out of range). Without the timeout, the ISR waits forever if there is no echo.

E1.6-03 — DHT22 Bit Decoding

Solution:

```
// (a) Decoding – threshold = 50  $\mu\text{s}$ :
pulses: [27, 71, 26, 68, 70]
bits: [ 0, 1, 0, 1, 1]

uint8_t decode_bit(uint16_t pulse_us) {
    return (pulse_us > 50) ? 1 : 0;
}

// (b) Threshold = 50  $\mu\text{s}$  (any value 29–69 works)

// (c) Worst-case 40 bits:
// Each bit: 50  $\mu\text{s}$  LOW + 70  $\mu\text{s}$  HIGH = 120  $\mu\text{s}$  maximum
// 40  $\times$  120 = 4800  $\mu\text{s}$  = 4.8 ms
// Plus start (800  $\mu\text{s}$ ) + response (80+80  $\mu\text{s}$ ): ~5.76 ms total
```

Explanation:

The 50 μs threshold gives 22 μs margin above the '0' maximum (28 μs) and 20 μs margin below the '1' minimum (70 μs) — maximum noise immunity. The full 40-bit DHT22 payload is: 16 bits humidity (MSB first) + 16 bits temperature + 8 bits checksum. Temperature bit 15 set means negative — negate the 15-bit value. Checksum = $(\text{hum_high} + \text{hum_low} + \text{temp_high} + \text{temp_low}) \& 0\text{xFF}$. If checksum fails, discard the entire reading and log an error counter.

E1.6-04 — Why Busy-Wait Breaks Under FreeRTOS

Solution:

Explanation:

A busy-wait loop counts iterations and assumes each iteration takes a fixed number of CPU cycles. Under FreeRTOS, the SysTick interrupt fires every 1 ms. If the scheduler decides to run UART_TX mid-loop, the CPU switches to UART_TX for up to several milliseconds, then returns to DHT22 bit reading. The 50 μs delay has become 5000+ μs . The DHT22's next bit is sampled hours late in protocol terms. Observable symptom: intermittent checksum failures or all-zero readings — not reproducible without FreeRTOS because the context switch only happens sometimes. Correct fix: Timer Input Capture. Hardware captures edge timestamps with zero CPU involvement and is completely immune to context switches. The ISR fires on each edge, stores the counter value, and the task reads 40 stored values after the full transmission is complete.

E1.6-05 — Input Capture ISR — HC-SR04

Solution:

```
static volatile uint16_t rising, falling;
static volatile uint8_t capture_done, waiting_fall;
```

```

void TIM3_IRQHandler(void) {
    if (TIM3->SR & (1U<<1)) { // CC1IF: capture fired
        TIM3->SR &= ~(1U<<1);
        if (!waiting_fall) {
            rising = TIM3->CCR1;
            waiting_fall = 1;
            TIM3->CCER |= (1U<<1); // CC1P=1 → capture falling edge
        } else {
            falling = TIM3->CCR1;
            waiting_fall = 0;
            TIM3->CCER &= ~(1U<<1); // restore rising edge
            uint16_t w = (falling >= rising) ? (falling - rising)
                : (0xFFFF - rising + falling + 1);
            capture_done = 1; // signal task
        }
    }

    if (TIM3->SR & (1U<<0)) { // UIF: overflow = timeout
        TIM3->SR &= ~(1U<<0);
        waiting_fall = 0;
        capture_done = 1; // signal timeout
    }
}

// In FreeRTOS: replace capture_done flag with xSemaphoreGiveFromISR()
// Task blocks on xSemaphoreTake() instead of polling the flag.

```

Explanation:

Toggling CC1P (polarity bit in CCER) between captures lets a single timer channel detect both rising and falling edges. The overflow interrupt (UIF) is the timeout handler — without it the ISR waits forever for a falling edge that never arrives when no object is detected. The counter overflow correction ($0xFFFF - \text{rising} + \text{falling} + 1$) handles the case where the counter wrapped from 65535 to 0 during the echo pulse. In FreeRTOS, replace the bare volatile flag with `xSemaphoreGiveFromISR()` — polling a flag wastes CPU; a semaphore lets the task block efficiently.

Solutions — Phase 2A: FreeRTOS Task Architecture

E2A-01 — Stack Size Estimation

Solution:

```
Call chain: 4 levels × 8 words = 32 words
Locals: 10 × uint32_t + char[32]/4 = 10 + 8 = 18 words
FreeRTOS: 64 words (context save area)
Subtotal: 32 + 18 + 64 = 114 words
Safety (×1.5): 114 × 1.5 = 171 words
Round to 64: 192 words = 768 bytes

#define STACK_SENSOR 192 // words passed to xTaskCreate

// Verify at runtime:
UBaseType_t hwm = uxTaskGetStackHighWaterMark(xSensorTask);
// hwm < 32 words → increase stack
```

Explanation:

`uxTaskGetStackHighWaterMark()` returns the minimum words that remained unused since the task started. A watermark below 32 words means the stack came dangerously close to overflowing. With `configCHECK_FOR_STACK_OVERFLOW=2`, FreeRTOS writes a known pattern at the stack base and checks it on every context switch — if corrupted, `vApplicationStackOverflowHook` fires with the task name. In production, measure watermarks under full system load (all sensors reading, UART busy, MQTT flowing) before finalising stack sizes.

E2A-02 — Priority Inversion — Mutex vs Semaphore

Solution:

Explanation:

Inversion sequence: 1. `SystemHealth` (LOW) takes I2C semaphore, starts reading a diagnostic register. 2. `SensorAcq` (HIGH) wakes, tries to take I2C semaphore — it is held by `SystemHealth`. `SensorAcq` BLOCKS. 3. `UART_TX` (MEDIUM) becomes the highest runnable task — `SensorAcq` is blocked, `SystemHealth` is preempted. `UART_TX` runs. 4. `SystemHealth` cannot run (`UART_TX` has higher priority). `SensorAcq` waits indefinitely for a LOW-priority task to finish. Mutex fix: FreeRTOS mutexes implement priority inheritance. When `SensorAcq` (HIGH) blocks on the mutex held by `SystemHealth` (LOW), FreeRTOS temporarily raises `SystemHealth`'s priority to HIGH. `SystemHealth` finishes and releases the mutex. `UART_TX` cannot preempt it during that window. After release, `SystemHealth` drops back to LOW.

E2A-03 — Queue Design for Sensor Data

Solution:

```
typedef struct {
    uint32_t timestamp_ms;
    int32_t bmp280_temp; // 0.01 °C
    uint32_t bmp280_pressure; // Pa
```

```

uint16_t dht22_humidity; // 0.1 %
int16_t dht22_temp; // 0.1 °C
uint16_t hcsr04_dist_mm;
} SensorReading; // 20 bytes

// Net accumulation at worst case: 10 - 6.7 = 3.3 items/sec
// Buffer 3 seconds of burst: 3.3 × 3 ≈ 10 items
QueueHandle_t q = xQueueCreate(10, sizeof(SensorReading));

// Producer on overflow – never block, log and continue:
if (xQueueSend(q, &reading;, 0) != pdTRUE) {
    drop_count++;
    // SystemHealth publishes drop_count to MQTT
}

```

Explanation:

Using portMAX_DELAY in xQueueSend would block SensorAcquisition when the queue is full, delaying the next sensor reading and cascading the problem. Non-blocking with logging is correct: drop the reading, increment a counter, continue sampling at the correct rate. The drop_count published to MQTT gives you visibility into queue saturation in production — if it climbs during normal operation your queue is too small or the consumer is too slow. Queue RAM = 10 × 20 = 200 bytes from the FreeRTOS Heap_4 pool.

E2A-04 — ISR-Safe API — The One Rule

Solution:

```

// WRONG:
void USART2_IRQHandler(void) {
    uint8_t b = USART2->DR;
    xQueueSend(CmdQueue, &b;, 0); // task API in ISR – WRONG
}

// WHY IT CRASHES: xQueueSend() calls taskENTER_CRITICAL() which
// manipulates the scheduler's uxSchedulerSuspended counter and
// pxCurrentTCB – these must only be touched from task context.
// Calling from ISR corrupts scheduler state. Crash only when ISR
// fires mid-context-switch → hence intermittent (hours to trigger).

// CORRECT:
void USART2_IRQHandler(void) {
    if (USART2->SR & (1U<<5)) {
        uint8_t b = (uint8_t)USART2->DR;
        BaseType_t woken = pdFALSE;
        xQueueSendFromISR(CmdQueue, &b;, &woken);
        portYIELD_FROM_ISR(woken);
        // portYIELD_FROM_ISR: if woken=pdTRUE a higher-priority task
        // was unblocked – sets PendSV to switch immediately on ISR exit.
    }
}

```

Explanation:

FromISR APIs never call taskSuspendAll, never block, and use BASEPRI (interrupt-masking) for their critical sections rather than PRIMASK — safe from interrupt context. portYIELD_FROM_ISR is critical for responsiveness: without it, a task unblocked by the ISR must wait up to 1 ms for the next SysTick scheduler tick before running. With it, PendSV triggers a context switch immediately on ISR exit — latency drops from up to 1 ms to microseconds. Rule: any FreeRTOS API inside an ISR must have FromISR in its name.

E2A-05 — vTaskDelay vs vTaskDelayUntil

Solution:

```
// (a) vTaskDelay(100):
// Actual cycle = 15ms work + 100ms delay = 115ms
// Rate = 1000/115 = 8.7 Hz
// After 60s: 60000/115 = 521 samples (should be 600 – 13% short)

void SensorTask_WRONG(void *p) {
    while(1) {
        read_sensors(); // 15 ms
        vTaskDelay(pdMS_TO_TICKS(100)); // delay AFTER
    }
}

// (b) vTaskDelayUntil(100ms period):
// Work runs inside the 100ms window – timing is absolute
// Rate = exactly 10.0 Hz
// After 60s: 600 samples (correct)

void SensorTask_CORRECT(void *p) {
    TickType_t last = xTaskGetTickCount();
    while(1) {
        vTaskDelayUntil(&last, pdMS_TO_TICKS(100)); // wake at absolute time
        read_sensors(); // 15 ms – doesn't affect next wake time
    }
}
```

Explanation:

vTaskDelay(N) delays N ticks from the point it is called — any work done before it adds to the effective period. vTaskDelayUntil(&last, N) delays until last + N ticks, updating last automatically. Work duration is absorbed within the period window. After 60 seconds with vTaskDelay: 521 samples instead of 600 — a 79-sample deficit that grows forever. With vTaskDelayUntil: 600 samples exactly. Use vTaskDelayUntil for any periodic task with a rate requirement. Exception: if work can occasionally exceed the period, vTaskDelayUntil fires back-to-back — add a guard that logs a warning if work_duration > period.

E2A-06 — Deadlock — Detection and Fix

Solution:

Explanation:

Deadlock sequence: 1. SensorAcq takes MutexA. 2. CommandHandler takes MutexB. 3. SensorAcq tries MutexB — blocked (CommandHandler holds it). 4. CommandHandler tries MutexA — blocked (SensorAcq holds it). Both

wait forever. Detection: SystemHealth calls uxTaskGetStackHighWaterMark() on both tasks every 10 seconds. If the watermark stops changing AND the SensorDataQueue shows no new items for >10 seconds, flag 'deadlock_suspected' on MQTT. Fix — global mutex ordering rule: always acquire mutexes in a fixed order (e.g., MutexA before MutexB, never the reverse). Wrap acquisition in a helper: take_i2c_then_uart(). This eliminates the circular wait condition — deadlock is impossible when all tasks acquire resources in the same order.

Solutions — Phase 2B: ESP32 + MQTT

E2B-01 — MQTT Topic Hierarchy Design

Solution:

```
// Telemetry (ESP32 publishes):
device/001/sensors/temperature // BMP280 temp in 0.01 °C
device/001/sensors/pressure // BMP280 in Pa
device/001/sensors/humidity // DHT22 in 0.1 %
device/001/sensors/distance // HC-SR04 in mm

// Health (ESP32 publishes every 10s):
device/001/health // JSON: {uptime,heap,wd_resets,rssi}

// OTA (laptop publishes, ESP32 subscribes):
device/001/ota/start // JSON: {size, crc32, version}
device/001/ota/chunk // {index, data[128]}
device/001/ota/status // ESP32 publishes: {ok|crc_fail|flash_err}

// Config (laptop publishes, STM32 side handles via ESP32 forward):
device/001/config/poll_interval // ms between sensor reads

// Laptop wildcard subscriptions:
device+/sensors/# // all sensor types from all nodes
device+/health // health from all nodes
```

Explanation:

Using {node_id} in the topic allows one broker to serve multiple sensor nodes without collision. The laptop subscribes to device+/sensors/# to receive all sensor data from all nodes with one subscription. QoS recommendations: sensor telemetry QoS 0 (dropped readings are acceptable at 10Hz), OTA start/chunk/status QoS 1 (must be received), config change QoS 1. Set the Last Will to device/001/status = 'offline' on connection, publish 'online' immediately after connect — this gives instant device-availability visibility on the dashboard.

E2B-02 — MQTT QoS Selection

Solution:

```
a) Sensor telemetry → QoS 0
10 Hz — next reading arrives in 100 ms. Drop cost is one missed sample.
QoS 1 ACK overhead would double MQTT traffic with no benefit.

b) OTA START → QoS 1
Must be received once. Duplicate is benign (OTA already in progress, ignored).

c) OTA DATA chunk → QoS 1
Each chunk must arrive. Sequence numbers detect and handle duplicates.

d) Health ping → QoS 0
Diagnostic only. Missed pings show as gaps in dashboard — acceptable.

e) Config change → QoS 1
Must be applied. Same config applied twice is idempotent — safe.
```

Explanation:

QoS 2 (exactly-once) costs 4 messages per delivery and should be used only when duplicates cause real problems AND QoS 1 with idempotent handling is insufficient. For IoT sensor nodes, QoS 2 is almost never necessary — design your handler to tolerate duplicates (check if value changed before applying) and use QoS 1 instead. The OTA protocol has its own reliability layer (sequence numbers, CRC, ACK/NAK) which makes QoS 2 redundant for OTA chunks.

E2B-03 — WiFi Reconnection Handler

Solution:

```
static int retry_delay_ms = 1000; // doubles on each failure, caps at 60000

static void wifi_handler(void *arg, esp_event_base_t base,
int32_t id, void *data) {
    if (base == WIFI_EVENT) {
        switch (id) {
            case WIFI_EVENT_STA_START:
                esp_wifi_connect();
                break;
            case WIFI_EVENT_STA_CONNECTED:
                retry_delay_ms = 1000; // reset backoff on success
                break;
            case WIFI_EVENT_STA_DISCONNECTED:
                ESP_LOGW(TAG, "Disconnected, retry in %d ms", retry_delay_ms);
                vTaskDelay(pdMS_TO_TICKS(retry_delay_ms));
                retry_delay_ms = (retry_delay_ms * 2 > 60000) ? 60000
                    : retry_delay_ms * 2;
                esp_wifi_connect();
                break;
        }
    } else if (base == IP_EVENT && id == IP_EVENT_STA_GOT_IP) {
        retry_delay_ms = 1000; // also reset on successful IP
        esp_mqtt_client_reconnect(mqtt_client); // reconnect MQTT
    }
}
```

Explanation:

Exponential backoff prevents hammering a recovering router with connection attempts. The 60-second cap ensures the device always retries within a minute of the router returning. MQTT reconnect belongs in IP_EVENT_STA_GOT_IP rather than WIFI_EVENT_STA_CONNECTED because IP assignment happens after WiFi association and may take an additional 100–300 ms. Reconnecting MQTT immediately after WiFi association (before IP) fails silently. Always log the retry delay — it appears in your SystemHealth MQTT topic and helps diagnose intermittent connectivity issues in production.

E2B-04 — Partition Table for OTA

Solution:

```
# partitions.csv
# Name, Type, SubType, Offset, Size
nvs, data, nvs, 0x9000, 0x6000, # 24 KB NVS storage
otadata, data, ota, 0xF000, 0x2000, # 8 KB OTA status
phy_init, data, phy, 0x11000, 0x1000, # 4 KB RF calibration
factory, app, factory, 0x20000, 0x100000, # 1 MB factory firmware
ota_0, app, ota_0, 0x120000, 0x140000, # 1.25 MB OTA slot 0
ota_1, app, ota_1, 0x260000, 0x140000, # 1.25 MB OTA slot 1
# Last address: 0x260000 + 0x140000 = 0x3A0000 = 3.6 MB < 4 MB ✓
# Verify firmware fits: idf.py size → check 'text+data' < 1.25 MB
```

Explanation:

The otadata partition stores which OTA slot is active and the pending update state — without it OTA does not work. The factory partition is the fallback: if both OTA slots fail verification on boot, ESP-IDF boots the factory image. OTA slot size = maximum firmware binary size — 1.25 MB supports most ESP-IDF applications. Run 'idf.py size' after building and verify the binary is well below 1.25 MB before ordering hardware. Changing the partition table after deployment requires erasing and re-flashing the device physically — design it correctly once.

Solutions — Phase 3: Inter-MCU Protocol Design

E3-01 — Frame Field Justification

Solution:

Explanation:

(a) LEN=2 bytes: 1 byte allows max 255-byte payload. OTA chunks may be 256 bytes and future payloads could be larger. 2 bytes (uint16) = 65535-byte max — future-proof at cost of 1 extra byte per frame. (b) SEQ=1 byte: at 10 Hz, 256 sequence numbers roll over in 25.6 seconds — well beyond the 500 ms ACK timeout. Within any ACK window the sequence is unique, so 1 byte is sufficient. (c) CMD=1 byte: 256 possible commands, current set is 12 — 244 spare codes for future use. (d) CRC=2 bytes: CRC-8 has only 256 possible values, 1-in-256 chance of corruption passing undetected. CRC-16 has 65536 values — 1-in-65535 chance. For firmware OTA, the lower false-pass probability is worth 1 extra byte. CRC-32 (4 bytes) would add more overhead per small frame than it saves for the frame's data size.

E3-02 — Byte Stuffing Encode and Decode

Solution:

```
// (a) Encode [0x01, 0xAA, 0x03, 0xBB, 0x55]:
0x01 → [0x01]
0xAA → [0xBB, 0x8A]
0x03 → [0x03]
0xBB → [0xBB, 0x9B]
0x55 → [0xBB, 0x75]
Encoded: [0x01, 0xBB, 0x8A, 0x03, 0xBB, 0x9B, 0xBB, 0x75]
5 bytes → 8 bytes (3 bytes added)

// (b) Decode [0x42, 0xBB, 0x8A, 0x15, 0xBB, 0x9B]:
0x42 → 0x42
0xBB, 0x8A → 0xAA (unescape)
0x15 → 0x15
0xBB, 0x9B → 0xBB (unescape)
Decoded: [0x42, 0xAA, 0x15, 0xBB]

// (c) Worst-case expansion:
// All bytes are 0xAA/0x55/0xBB → every byte doubles
// 255-byte payload → up to 510-byte encoded
// Ratio: 2:1 (average with random data: ~1% expansion)
```

Explanation:

The escape byte 0xBB itself must be escaped — otherwise a 0xBB in the payload followed by any of the special second bytes would be misinterpreted by the decoder. The XOR-0x20 pattern (0xAA^0x20=0x8A, 0x55^0x20=0x75, 0xBB^0x20=0x9B) is self-consistent: the decoder sees 0xBB, reads the next byte, XORs with 0x20 to recover the original. Worst-case 2:1 expansion means a 255-byte payload could become 510 bytes after stuffing — this affects UART transfer time. The LEN field in the frame header should reflect the pre-stuffed payload length so the receiver knows exactly how many decoded bytes to expect.

E3-03 — CRC-16/CCITT Manual Calculation

Solution:

```
Initial CRC = 0xFFFF

After byte 0x01 (bit-by-bit):
Bit 7(0): MSB=1, 1 XOR 0=1 → (0xFFFE ^ 0x1021) = 0xEFDF
Bit 6(0): MSB=1, 1 XOR 0=1 → (0xDFBE ^ 0x1021) = 0xCF9F
[... 6 more bits ...]
After 0x01: CRC = 0xF1D1

After byte 0x02: CRC = 0x4503
After byte 0x03: CRC = 0x6B56

// Verify at crccalc.com:
// Input "010203" hex, select CRC-16/CCITT → 0x6B56 ✓
```

Explanation:

The table-driven implementation precomputes this 8-bit-at-a-time process for all 256 byte values and stores results in a 512-byte table. Instead of 8 iterations per byte, it does one table lookup plus one XOR — 8× faster. The manual calculation proves the table contains correct values. CRC-16/CCITT parameters: polynomial=0x1021, init=0xFFFF, RefIn=false, RefOut=false, XorOut=0x0000. Any deviation in parameters produces a completely different CRC — both ends must use identical parameters.

E3-04 — Parser Timeout and Recovery

Solution:

```
// (a) Without timeout: parser stays in ST_PAYLOAD forever.
// No new frames can be decoded because the parser never returns to IDLE.

// (b) Timeout check (call from UARTReceive task every 10 ms):
void parser_timeout_check(ProtocolParser *p) {
    if (p->state != ST_IDLE) {
        uint32_t elapsed = systick_get_ms() - p->last_byte_ms;
        if (elapsed > 500) {
            uart_printf("[PROTO] timeout in state %d after %lu ms\r\n",
                p->state, elapsed);
            parser_reset(p);
        }
    }
}

// (c) Reset to ST_IDLE, clear all:
void parser_reset(ProtocolParser *p) {
    p->state = ST_IDLE;
    p->payload_idx = 0;
    p->payload_len = 0;
    p->seq = 0;
    p->cmd = 0;
    p->crc_rx = 0;
```

```
memset(p->payload, 0, sizeof(p->payload));  
// keep last_byte_ms – it records when the timeout fired  
}
```

Explanation:

Without timeout, a partial frame permanently corrupts parser state. The next valid frame's SOF byte (0xAA) arrives and is interpreted as the 11th payload byte — it corrupts that slot and the parser stays in ST_PAYLOAD expecting more bytes that never come. The 500 ms timeout is chosen based on maximum inter-byte gap at 115200 baud: a 255-byte frame takes ~22 ms. Any gap over 500 ms indicates the frame was abandoned. After reset to ST_IDLE the next SOF byte immediately starts a new frame. Log persistent timeouts — they indicate a wiring or electrical noise problem.

Solutions — Phase 4: Bootloader + OTA Pipeline

E4-01 — Flash Sector Address Arithmetic

Solution:

```
// (a) Last byte of Sector 0:
0x08000000 + 0x4000 - 1 = 0x08003FFF

// (b) 256th byte of application firmware:
// App starts at 0x08010000
0x08010000 + 256 - 1 = 0x080100FF

// (c) Application region size:
// Sector 4: 64 KB = 65,536 bytes
// Sectors 5,6,7: 3 × 128 KB = 393,216 bytes
// Total: 458,752 bytes
87,420 < 458,752 → YES, fits with 371,332 bytes spare.

// Sectors needed:
// Sector 4 holds 65,536 bytes of the 87,420-byte firmware.
// Remaining: 87,420 - 65,536 = 21,884 bytes → start of Sector 5.
// Sectors to erase on OTA update: Sector 4 + part of Sector 5 = erase both.
```

Explanation:

Always erase complete sectors — you cannot erase partial sectors. Even if only 21,884 bytes of Sector 5 are used, you erase the full 128 KB sector before writing. On OTA update: erase Sector 4 then Sector 5, write 87,420 bytes, compute CRC over the written region, compare to expected CRC. Your OTA handler must guard against writing to Sector 0 (bootloader) — add an explicit check: `if (target_addr < 0x08010000) { abort; }`

E4-02 — Linker Script Bug — Wrong ORIGIN

Solution:

```
// BUG: FLASH ORIGIN = 0x08000000 – wrong for the application.
// The linker resolved all symbols assuming code runs from 0x08000000.
// But the bootloader flashed the application to 0x08010000.
// Every function call address is 0x10000 bytes too low.

// FAILURE MECHANISM:
// 1. Bootloader jumps to 0x08010000 (correct – reset handler is there)
// 2. Reset handler runs, calls SystemInit()
// 3. Linker compiled SystemInit() as if it lives at ~0x08001234
// but it's actually at 0x08011234 (flash + 0x10000)
// 4. Branch instruction targets 0x08001234 – inside the BOOTLOADER
// 5. Bootloader code runs in application context → HardFault

// CORRECT application linker script:
MEMORY {
FLASH (rx) : ORIGIN = 0x08010000, LENGTH = 448K
RAM (rwx) : ORIGIN = 0x20000000, LENGTH = 128K
```

```

}

// Also add to SystemInit() in application:
// SCB->VTOR = 0x08010000; // relocate vector table to app region

```

Explanation:

The VTOR (Vector Table Offset Register) must also point to the application's vector table at 0x08010000. Without this, all interrupts use the bootloader's vector table entries, causing wrong interrupt handlers to execute. These two changes — linker ORIGIN and VTOR — are always made together. Without VTOR: your application timer interrupt calls the bootloader's SysTick handler, which probably no longer exists or jumps to garbage. To verify: after flashing, read SCB->VTOR via GDB and confirm it equals 0x08010000.

E4-03 — Boot Jump Sequence — Find Missing Steps

Solution:

```

void jump(uint32_t app) {
    __disable_irq(); // Step 1: mask interrupts ✓

    // Step 2: MISSING – stop SysTick
    // Without this, SysTick fires after VTOR update and calls
    // the application's SysTick handler before .data/.bss are initialised
    SysTick->CTRL = 0;
    SysTick->LOAD = 0;
    SysTick->VAL = 0;

    SCB->VTOR = app; // Step 3: relocate vector table ✓

    // Step 4: MISSING – pipeline flush
    // Without DSB+ISB, CPU may cache old VTOR for pending interrupts
    __DSB();
    __ISB();

    // Step 5: MISSING – load application stack pointer
    // Without this, application writes over bootloader's stack on first push
    __set_MSP(*(volatile uint32_t *)app);

    // Step 6+7: get reset handler and jump ✓
    void (*reset)(void) = (void(*) (void))(*(volatile uint32_t *) (app+4));
    reset();
}

```

Explanation:

Missing SysTick stop: if SysTick fires between VTOR update and jump it calls the application's SysTick handler before the startup code has initialised .data (copied from flash) and .bss (zeroed). Any global variable access in the handler crashes. Missing DSB+ISB: Data Synchronisation Barrier and Instruction Synchronisation Barrier ensure the VTOR write propagates through the memory system and the CPU pipeline is flushed before executing the next instruction. Without them, the CPU may still use the old VTOR value cached in its pipeline. Missing MSP load: the application's initial stack pointer is the first 32-bit word of its vector table. Without loading it, the application's first stack push goes to the bootloader's stack region and corrupts bootloader data that may still be in RAM.

E4-04 — OTA Transfer Timeline

Solution:

```
// (a) Chunks:
full = 87420 / 128 = 682.96 → 682 full chunks + 1 partial
last_chunk = 87420 - 682×128 = 87420 - 87296 = 124 bytes
Total chunks = 683

// (b) Per-chunk frame time (136 bytes worst case with stuffing):
frame_bytes = 1+2+1+1+128+2+1 = 136 bytes
bit_time = 10 / 115200 = 86.8 μs per byte
frame_time = 136 × 86.8 μs = 11.8 ms per chunk

// (c) Total no ACK:
683 × 11.8 ms = 8059 ms ≈ 8.1 seconds

// (d) With ACK per chunk (10ms RTT):
ACK_frame = 7 bytes × 86.8 μs = 0.6 ms
per_chunk = 11.8 + 0.6 + 10 = 22.4 ms
Total = 683 × 22.4 ms = 15,299 ms ≈ 15.3 seconds
```

Explanation:

15 seconds for a complete OTA update is acceptable for an infrequent operation. Set your OTA timeout at 30 seconds minimum to account for occasional retransmissions and MQTT delivery delays. If you need faster updates: increase UART baud rate to 460800 (4× faster, halves transfer time), or increase chunk size to 256 bytes (reduces per-chunk overhead ratio). The ACK-per-chunk scheme adds 10ms overhead per chunk but guarantees every chunk is received before the next is sent — simple and reliable for a first implementation. A sliding-window scheme (acknowledge every N chunks) reduces overhead at the cost of more complex error recovery.

E4-05 — Flash Write with Verification

Solution:

```
typedef enum { FLASH_OK=0, ERR_WRP, ERR_PGA, ERR_PGP, ERR_PGS,
ERR_VERIFY, ERR_TIMEOUT } FlashError;

FlashError flash_write_verified(uint32_t addr, uint32_t data) {
FLASH->KEYR = 0x45670123; // unlock key 1
FLASH->KEYR = 0xCDEF89AB; // unlock key 2

FLASH->CR &= ~(0x3U << 8); // clear PSIZE
FLASH->CR |= (0x2U << 8); // PSIZE=10 → x32 writes at 2.7–3.6V
FLASH->CR |= (1U << 0); // PG=1 enable programming

*(volatile uint32_t *)addr = data; // write

uint32_t t0 = systick_get_ms();
while (FLASH->SR & (1U << 16)) { // wait BSY
if (systick_get_ms() - t0 > 1000) { goto lock; return ERR_TIMEOUT; }
}

FlashError e = FLASH_OK;
if (FLASH->SR & (1U<<4)) e = ERR_WRP;
if (FLASH->SR & (1U<<5)) e = ERR_PGA;
if (FLASH->SR & (1U<<6)) e = ERR_PGP;
```

```
if (FLASH->SR & (1U<<7)) e = ERR_PGS;
if (e != FLASH_OK) goto lock;

if (*(volatile uint32_t *)addr != data) { e = ERR_VERIFY; }

lock:
FLASH->CR &= ~(1U << 0); // PG=0
FLASH->CR |= (1U << 31); // LOCK
return e;
}
```

Explanation:

PSIZE (parallelism size) must match supply voltage: at 2.7–3.6V (which your 3.3V system provides) use x32 (PSIZE=10) for 32-bit word writes. Writing with wrong PSIZE causes PGPERR. Clear error flags before writing by writing 1 to them in FLASH_SR — residual flags from a previous operation would be misread as new errors from this write. The read-back verify step is the definitive check: even if no status flags are set, a marginal flash cell might not retain the value. In production bootloaders the full CRC of the written image is computed after all sectors are written, catching any verify failure comprehensively.
